Ağ Çalışma Grubu T. Berners-Lee Yorum Talebi: 1945 MIT/LCS Kategori: Bilgilendirici R. Fielding UC Irvine H. Frystyk MIT/LCS Mayıs 1996

Hypertext Transfer Protocol -- HTTP/1.0

Bu Memonun Durumu

Bu memo, İnternet topluluğuna bilgi sunmak amacıyla hazırlanmıştır. Herhangi bir İnternet standardı belirtmemektedir. Bu memonun dağıtımı sınırsızdır.

IESG Notu:

IESG bu protokol konusunda endişe taşımakta ve bu belgenin yakın zamanda bir standartlar izleme belgesiyle değiştirileceğini öngörmektedir.

Özet

Hypertext Transfer Protocol (HTTP), dağıtık, işbirlikçi, hypermedia bilgi sistemleri için gereken hafiflik ve hızı bünyesinde barındıran uygulama düzeyinde bir protokoldür. Request methodlarının (komutların) genişletilmesiyle isim sunucuları ve dağıtık nesne yönetim sistemleri gibi pek çok görev için kullanılabilen genel amaçlı, durumsuz, nesne yönelimli bir protokoldür. HTTP'nin bir özelliği de veri gösterimini tiplere ayırmasıdır; bu sayede sistemler, aktarılan veriden bağımsız olarak inşa edilebilir.

HTTP, 1990'dan bu yana World-Wide Web küresel bilgi girişimi tarafından kullanılmaktadır. Bu spesifikasyon, "HTTP/1.0" olarak anılan protokolün yaygın kullanımını yansıtmaktadır.

İçindekiler

Giriş 4

1.1 Amaç 4

1.2 Terminoloji 4

1.3 Genel İşleyiş 6

1.4 HTTP ve MIME 8

Gösterim Kuralları ve Genel Gramer 8

2.1 Genişletilmiş BNF 8

2.2 Temel Kurallar 10

Protokol Parametreleri 12

3.1 HTTP Versiyonu 12

3.2 Uniform Resource Identifier'lar 14

3.2.1 Genel Sözdizimi 14

3.2.2 http URL 15

3.3 Tarih/Saat Formatları 15

3.4 Karakter Setleri 17

3.5 Content Coding'ler 18

3.6 Media Type'lar 19

3.6.1 Kanonikalleştirme ve Metin Varsayılanları 19

3.6.2 Multipart Type'lar 20

3.7 Ürün Token'ları 20

HTTP Message 21

4.1 Message Type'ları 21

4.2 Message Header'ları 22

4.3 Genel Header Field'ları 23

Request 23

5.1 Request-Line 23

5.1.1 Method 24

5.1.2 Request-URI 24

5.2 Request Header Field'ları 25

Response 25

6.1 Status-Line 26

6.1.1 Status Code ve Reason Phrase 26

6.2 Response Header Field'ları 28

Entity 28

7.1 Entity Header Field'ları 29

7.2 Entity Body 29

7.2.1 Type 29

7.2.2 Length 30

Method Tanımları 30

8.1 GET 31

8.2 HEAD 31

8.3 POST 31

---

1. *Giriş*

1.1 Amaç

Hypertext Transfer Protocol (HTTP), dağıtık, işbirlikçi, hypermedia bilgi sistemleri için gereken hafiflik ve hızı bünyesinde barındıran uygulama düzeyinde bir protokoldür. HTTP, 1990'dan bu yana World-Wide Web küresel bilgi girişimi tarafından kullanılmaktadır. Bu spesifikasyon, "HTTP/1.0" olarak anılan protokolün yaygın kullanımını yansıtmaktadır. Bu spesifikasyon, HTTP/1.0 client ve server'larının büyük çoğunluğunda tutarlı biçimde hayata geçirildiği görülen özellikleri tanımlamaktadır. Spesifikasyon iki bölüme ayrılmıştır. HTTP'nin genellikle tutarlı biçimde gerçeklenen özellikleri bu belgenin ana gövdesinde açıklanmış; az sayıda ya da tutarsız gerçeklemesi bulunan özellikler ise Ek D'de listelenmiştir.

Pratik bilgi sistemleri, basit erişimin ötesinde arama, ön uç güncelleme ve açıklama ekleme gibi ek işlevsellik gerektirir. HTTP, bir request'in amacını belirtmek için açık uçlu bir method kümesinin kullanılmasına olanak tanır. Bir method'un uygulanacağı kaynağı belirtmek için konum (URL) [4] veya ad (URN) [16] olarak Uniform Resource Identifier (URI) [2] tarafından sağlanan referans disiplinine dayanır. Message'lar, İnternet Mail [7] ve Multipurpose Internet Mail Extensions (MIME) [5] tarafından kullanılana benzer bir formatta iletilir.

HTTP aynı zamanda, user agent'lar ile SMTP [12], NNTP [11], FTP [14], Gopher [1] ve WAIS [8] gibi diğer İnternet protokollerine açılan proxy/gateway'ler arasındaki iletişim için genel amaçlı bir protokol olarak da kullanılmakta; çeşitli uygulamalarda mevcut kaynaklara temel hypermedia erişimini sağlamakta ve user agent'ların gerçeklenmesini kolaylaştırmaktadır.

1.2 Terminoloji

Bu spesifikasyon, HTTP iletişimindeki katılımcıların ve nesnelerin rollerini tanımlamak için bir dizi terim kullanmaktadır.

connection (bağlantı) İletişim amacıyla iki uygulama programı arasında kurulan transport katmanı sanal devresi.

message Bölüm 4'te tanımlanan sözdizimiyle eşleşen ve connection üzerinden iletilen yapılandırılmış bir oktet dizisinden oluşan, HTTP iletişiminin temel birimi.

request Bir HTTP request message'ı (Bölüm 5'te tanımlandığı şekilde).

response Bir HTTP response message'ı (Bölüm 6'da tanımlandığı şekilde).

resource (kaynak) URI ile tanımlanabilen bir ağ veri nesnesi veya servisi (Bölüm 3.2).

entity Bir request veya response message içinde yer alabilen, bir veri kaynağının belirli bir gösterimi ya da bir servis kaynağından gelen yanıt. Bir entity, entity header'ları biçiminde metabilgiden ve entity body biçiminde içerikten oluşur.

client Request göndermek amacıyla connection kuran uygulama programı.

user agent Request'i başlatan client. Bunlar çoğunlukla tarayıcılar, editörler, spider'lar (web'i dolaşan robotlar) veya diğer son kullanıcı araçlarıdır.

server Request'lere response göndererek onları karşılamak için connection kabul eden uygulama programı.

origin server Belirli bir kaynağın bulunduğu ya da oluşturulacağı server.

proxy Diğer client'lar adına request yapmak amacıyla hem server hem de client olarak işlev gören aracı program. Request'ler dahili olarak karşılanır ya da olası çeviriyle birlikte başka server'lara iletilir. Bir proxy, bir request message'ı iletmeden önce yorumlamalı ve gerekirse yeniden yazmalıdır. Proxy'ler çoğunlukla ağ firewall'ları üzerinden client tarafı portal'ları olarak ve user agent tarafından gerçeklenmeyen protokoller aracılığıyla request'leri işleyen yardımcı uygulamalar olarak kullanılır.

gateway Başka bir server için aracı olarak işlev gören server. Proxy'nin aksine, gateway request'leri sanki istenen kaynağın origin server'ıymış gibi alır; request'i yapan client, bir gateway ile iletişim kurduğunun farkında olmayabilir. Gateway'ler çoğunlukla ağ firewall'ları üzerinden server tarafı portal'ları olarak ve HTTP dışı sistemlerde depolanan kaynaklara erişim için protokol çevirici olarak kullanılır.

tunnel İki connection arasında kör bir aktarıcı olarak işlev gören aracı program. Bir kez aktif hale geldikten sonra tunnel, bir HTTP request'i tarafından başlatılmış olsa bile HTTP iletişiminin tarafı sayılmaz. Aktarılan her iki connection da kapandığında tunnel sona erer. Tunnel'lar, bir portal gerekli olduğunda ve aracının aktarılan iletişimi yorumlayamadığı ya da yorumlamaması gerektiği durumlarda kullanılır.

cache Response message'larının program içindeki yerel deposu ve bu message'ların depolanması, alınması ile silinmesini denetleyen alt sistem. Cache, gelecekteki eşdeğer request'lerde response süresini ve ağ bant genişliği tüketimini azaltmak için önbelleğe alınabilir response'ları saklar. Herhangi bir client veya server cache içerebilir; ancak bir server tunnel olarak işlev görürken cache kullanamaaz.

Herhangi bir program hem client hem de server olabilecek kapasitede olabilir; bu terimlerin kullanımı yalnızca programın belirli bir connection için üstlendiği role, programın genel yeteneklerine değil, atıfta bulunur. Benzer şekilde, herhangi bir server origin server, proxy, gateway veya tunnel olarak işlev görebilir; davranışı her request'in niteliğine göre değişir.

1.3 Genel İşleyiş

HTTP protokolü, bir request/response paradigmasına dayanır. Bir client, bir server ile connection kurar ve server'a request method, URI ve protokol versiyonu biçiminde, ardından request modifier'larını, client bilgilerini ve olası body içeriğini barındıran MIME benzeri bir message ile bir request gönderir. Server, message'ın protokol versiyonunu ve bir başarı ya da hata kodunu içeren bir status line ile; ardından server bilgilerini, entity metabilgisini ve olası body içeriğini barındıran MIME benzeri bir message ile yanıt verir.

HTTP iletişiminin büyük çoğunluğu bir user agent tarafından başlatılır ve bir origin server üzerindeki bir kaynağa uygulanacak bir request'ten oluşur. En basit durumda bu, user agent (UA) ile origin server (O) arasında tek bir connection (v) aracılığıyla gerçekleştirilebilir.

request zinciri ----------> UA ----------v---------- O <---------- response zinciri

Request/response zincirinde bir veya daha fazla aracının bulunması durumunda daha karmaşık bir durum ortaya çıkar. Üç yaygın aracı türü vardır: proxy, gateway ve tunnel. Proxy, request'leri mutlak biçimde URI olarak alıp message'ın tamamını veya bir bölümünü yeniden yazarak URI ile belirlenen server'a yönlendiren bir iletme aracısıdır. Gateway, diğer server'ların üzerinde bir katman olarak işlev gören ve gerektiğinde request'leri alttaki server protokolüne çeviren bir alıcı aracıdır. Tunnel, message'ları değiştirmeksizin iki connection arasında bir aktarma noktası olarak işlev görür; tunnel'lar, iletişimin bir aracıdan (güvenlik duvarı gibi) geçmesi gerektiğinde, aracı message içeriğini anlayamasa bile kullanılır.

request zinciri ----------> UA -----v----- A -----v----- B -----v----- C -----v----- O <---------- response zinciri

Yukarıdaki şekil, user agent ile origin server arasındaki üç aracıyı (A, B ve C) göstermektedir. Tüm zinciri kat eden bir request veya response message'ı dört ayrı connection'dan geçmek zorundadır. Bu ayrım önemlidir; zira bazı HTTP iletişim seçenekleri yalnızca en yakın tunnel olmayan komşuyla olan connection'a, yalnızca zincirin uç noktalarına ya da zincir boyunca tüm connection'lara uygulanabilir. Diyagram doğrusal olsa da her katılımcı aynı anda birden fazla iletişimde yer alıyor olabilir. Örneğin B, A'nın request'ini işlerken aynı zamanda A dışındaki pek çok client'tan request alıyor ve/veya C dışındaki server'lara request iletiyor olabilir.

Tunnel olarak işlev görmeyen iletişimin herhangi bir tarafı, request'leri işlemek için dahili bir cache kullanabilir. Cache'in etkisi, zincirdeki katılımcılardan birinin söz konusu request için geçerli önbelleğe alınmış bir response'a sahip olması durumunda request/response zincirinin kısalmasıdır. Aşağıda, B'nin UA ya da A tarafından önbelleğe alınmamış bir request için O'dan (C aracılığıyla) aldığı daha önceki bir response'un önbelleğe alınmış kopyasına sahip olduğu durumdaki sonuç zinciri gösterilmektedir.

request zinciri ----------> UA -----v----- A -----v----- B - - - - - - C - - - - - - O <--------- response zinciri

Tüm response'lar önbelleğe alınabilir değildir ve bazı request'ler, cache davranışına ilişkin özel gereksinimler içeren modifier'lar barındırabilir. Bazı HTTP/1.0 uygulamaları, bir response'un "önbelleğe alınabilir" olup olmadığını tanımlamak için buluşsal yöntemler kullanmaktadır; ancak bu kurallar standart hale getirilmemiştir.

İnternette HTTP iletişimi genellikle TCP/IP connection'ları üzerinden gerçekleşir. Varsayılan port TCP 80'dir [15]; ancak başka portlar da kullanılabilir. Bu durum, HTTP'nin İnternet'teki başka bir protokol veya başka ağlar üzerinde gerçeklenmesini engellemez. HTTP yalnızca güvenilir bir transport varsayar; bu güvenceleri sağlayan herhangi bir protokol kullanılabilir ve HTTP/1.0 request ile response yapılarının ilgili protokolün transport veri birimlerine eşlenmesi bu spesifikasyonun kapsamı dışındadır.

Deneysel uygulamalar dışında, mevcut pratik her request'ten önce connection'ın client tarafından kurulmasını ve response gönderildikten sonra server tarafından kapatılmasını gerektirmektedir. Hem client'lar hem de server'lar, taraflardan herhangi birinin kullanıcı eylemi, otomatik zaman aşımı veya program hatası nedeniyle connection'ı erken kapatabileceğinin farkında olmalı ve böyle bir kapanmayı öngörülebilir biçimde ele almalıdır. Her durumda, connection'ın bir veya her iki taraf tarafından kapatılması, durumu ne olursa olsun mevcut request'i her zaman sonlandırır.

1.4 HTTP ve MIME

HTTP/1.0, RFC 1521 [5]'te tanımlandığı üzere MIME için tanımlanmış yapıların büyük bölümünü kullanır. Ek C, HTTP bağlamının Internet Media Type'larının İnternet postasında tipik olarak bulunan kullanımdan farklı biçimlerde kullanılmasına nasıl olanak tanıdığını ve bu farklılıkların gerekçelerini açıklamaktadır.

2. Gösterim Kuralları ve Genel Gramer

2.1 Genişletilmiş BNF

Bu belgede belirtilen tüm mekanizmalar, hem düz metin olarak hem de RFC 822 [7] tarafından kullanılana benzer genişletilmiş bir Backus-Naur Form (BNF) ile tanımlanmaktadır. Gerçekleyicilerin bu spesifikasyonu anlayabilmek için söz dizimine aşina olmaları gerekmektedir. Genişletilmiş BNF aşağıdaki yapıları içerir:

name = definition (ad = tanım) Bir kuralın adı, açılı parantezler ("<" ve ">") olmaksızın yalnızca adın kendisidir ve tanımından "=" karakteriyle ayrılır. Boşluk yalnızca devam satırlarının girintilenmesinde anlamlıdır; bu, birden fazla satıra yayılan bir kural tanımını belirtmek için kullanılır. SP, LWS, HT, CRLF, DIGIT, ALPHA gibi bazı temel kurallar büyük harfle yazılır. Açılı parantezler, tanımlar içinde kural adlarının kullanımını ayırt etmeye yardımcı olduğu durumlarda kullanılır.

"literal" (değişmez) Tırnak işaretleri değişmez metni çevreler. Aksi belirtilmedikçe metin büyük/küçük harf duyarsızdır.

rule1 | rule2 (kural1 | kural2) Çubukla ayrılmış öğeler seçeneklerdir; örneğin "yes | no" ifadesi yes ya da no değerini kabul eder.

(rule1 rule2) (kural1 kural2) Parantez içindeki öğeler tek bir öğe olarak değerlendirilir. Örneğin "(elem (foo | bar) elem)", "elem foo elem" ve "elem bar elem" token dizilerine izin verir.

rule (kural) Bir öğenin önündeki "" karakteri tekrarı belirtir. Tam biçim "<n><m>element" şeklinde olup öğenin en az <n>, en fazla <m> kez geçtiğini belirtir. Varsayılan değerler 0 ve

*sonsuz olduğundan "(element)" sıfır dahil herhangi bir sayıya, "1element" en az bir tekrara ve "1*2element" bir ya da iki tekrara izin verir.*

*[rule] ([kural]) Köşeli parantezler isteğe bağlı öğeleri çevreler; "[foo bar]" ifadesi "*1(foo bar)" ile eşdeğerdir.*

N rule (N kural) Belirli tekrar: "<n>(element)", "<n><n>(element)" ile eşdeğerdir; yani (element)'in tam olarak <n> kez geçmesi anlamına gelir. Bu nedenle 2DIGIT iki basamaklı bir sayıyı, 3ALPHA ise üç alfabetik karakterden oluşan bir dizeyi ifade eder.*

#rule (#kural) Liste tanımlamak için ""a benzer bir "#" yapısı tanımlanmıştır. Tam biçim "<n>#<m>element" şeklinde olup her biri bir veya daha fazla virgül (",") ve isteğe bağlı doğrusal boşluk (LWS) ile ayrılmış en az <n>, en fazla <m> öğeyi belirtir. Bu, liste yazımını kolaylaştırır; "(*LWS element *(*LWS "," *LWS element))" gibi bir kural "1#element" olarak gösterilebilir. Bu yapının kullanıldığı her yerde boş öğelere izin verilir, ancak bunlar öğe sayısına dahil edilmez. Yani "(element), , (element)" geçerlidir, ancak yalnızca iki öğe olarak sayılır. Bu nedenle en az bir öğenin gerekli olduğu durumlarda en az bir boş olmayan öğe bulunmalıdır. Varsayılan değerler 0 ve sonsuz olduğundan "#(element)" sıfır dahil herhangi bir sayıya, "1#element" en az bir öğeye ve "1#2element" bir ya da iki öğeye izin verir.*

; comment (; yorum) Kural metninin sağında bir miktar boşluk bırakılarak yazılan noktalı virgül, satır sonuna kadar devam eden bir yorum başlatır. Bu, spesifikasyonlarla paralel olarak yararlı notlar eklemenin basit bir yoludur.

*implied *LWS (örtük *LWS) Bu spesifikasyonun tanımladığı gramer kelime tabanlıdır. Aksi belirtilmedikçe, bir alanın yorumunu değiştirmeksizin herhangi iki bitişik kelime (token veya quoted-string) arasına ve bitişik token'lar ile sınırlayıcılar (tspecials) arasına doğrusal boşluk (LWS) eklenebilir. İki token arasında en az bir sınırlayıcı (tspecials) bulunmalıdır; aksi takdirde tek bir token olarak yorumlanırlar. Bununla birlikte, uygulamalar HTTP yapılarını oluştururken "yaygın biçimi" takip etmeye çalışmalıdır; zira bazı gerçeklemeler yaygın biçimlerin ötesinde herhangi bir şeyi kabul etmemektedir.*

2.2 Temel Kurallar

Aşağıdaki kurallar, temel ayrıştırma yapılarını tanımlamak için bu spesifikasyon boyunca kullanılmaktadır. US-ASCII kodlu karakter seti [17] tarafından tanımlanmaktadır.

OCTET = <herhangi bir 8-bit veri dizisi> CHAR = <herhangi bir US-ASCII karakteri (oktet 0 - 127)> UPALPHA = <herhangi bir US-ASCII büyük harf "A".."Z"> LOALPHA = <herhangi bir US-ASCII küçük harf "a.."z"> ALPHA = UPALPHA | LOALPHA DIGIT = <herhangi bir US-ASCII rakamı "0".."9"> CTL = <herhangi bir US-ASCII kontrol karakteri (oktet 0 - 31) ve DEL (127)> CR = <US-ASCII CR, satır başı (13)> LF = <US-ASCII LF, satır besleme (10)> SP = <US-ASCII SP, boşluk (32)> HT = <US-ASCII HT, yatay sekme (9)> <"> = <US-ASCII çift tırnak işareti (34)>

HTTP/1.0, Entity-Body dışındaki tüm protokol öğeleri için satır sonu işaretleyicisi olarak CR LF oktet dizisini tanımlar (toleranslı uygulamalar için bkz. Ek B). Entity-Body içindeki satır sonu işaretleyicisi, Bölüm 3.6'da açıklandığı üzere ilgili media type tarafından tanımlanır.

CRLF = CR LF

HTTP/1.0 header'ları, her devam satırı bir boşluk veya yatay sekme ile başlıyorsa birden fazla satıra katlanabilir. Katlama dahil tüm doğrusal boşluklar, SP ile aynı anlambilimsel değere sahiptir.

LWS = [CRLF] 1(SP | HT)*

Ancak header satırlarının katlanması bazı uygulamalar tarafından beklenmemektedir ve HTTP/1.0 uygulamaları tarafından üretilmemelidir.

*TEXT kuralı yalnızca message ayrıştırıcısı tarafından yorumlanması amaçlanmayan açıklayıcı alan içerikleri ve değerleri için kullanılır. *TEXT sözcükleri US-ASCII dışındaki karakter setlerinden oktetler içerebilir.*

TEXT = <CTL'ler dışındaki herhangi bir OCTET, LWS dahil>

US-ASCII karakter setinin dışındaki oktetler içeren header field TEXT'ini alan alıcılar, bunların ISO-8859-1 karakterlerini temsil ettiğini varsayabilir.

Onaltılık sayısal karakterler çeşitli protokol öğelerinde kullanılır.

HEX = "A" | "B" | "C" | "D" | "E" | "F" | "a" | "b" | "c" | "d" | "e" | "f" | DIGIT

Pek çok HTTP/1.0 header field değeri, LWS veya özel karakterlerle ayrılmış sözcüklerden oluşur. Bu özel karakterlerin bir parametre değeri içinde kullanılabilmesi için quoted-string içinde yer alması gerekir.

word = token | quoted-string token = 1<CTL veya tspecials dışındaki herhangi bir CHAR> tspecials = "(" | ")" | "<" | ">" | "@" | "," | ";" | ":" | "" | <"> | "/" | "[" | "]" | "?" | "=" | "{" | "}" | SP | HT*

Bazı HTTP header field'larına, yorum metnini parantezle çevreleyerek yorum eklenebilir. Yorumlara yalnızca field değeri tanımının bir parçası olarak "comment" içeren field'larda izin verilir. Diğer tüm field'larda parantezler field değerinin bir parçası olarak kabul edilir.

*comment = "(" *(ctext | comment) ")" ctext = <"(" ve ")" dışındaki herhangi bir TEXT>*

Çift tırnak işaretiyle alıntılanan bir metin dizisi tek bir sözcük olarak ayrıştırılır.

*quoted-string = (<"> *(qdtext) <">) qdtext = <"> ve CTL'ler dışındaki herhangi bir CHAR, LWS dahil>*

Ters eğik çizgi ("\") karakteri kullanılarak tek karakter alıntılanması HTTP/1.0'da desteklenmez.

3. *Protokol Parametreleri*

3.1 HTTP Versiyonu

HTTP, protokolün versiyonlarını belirtmek için "<major>.<minor>" numaralandırma şemasını kullanır. Protokol versiyonlama politikası, gönderenin bir message'ın formatını ve daha ileri HTTP iletişimini anlama kapasitesini, bu iletişim aracılığıyla elde edilen özellikleri değil, belirtmesine izin vermeyi amaçlamaktadır. İletişim davranışını etkilemeyen veya yalnızca genişletilebilir field değerlerine ekleme yapan message bileşenlerinin eklenmesi için versiyon numarasında herhangi bir değişiklik yapılmaz.

<minor> numarası, protokolde yapılan değişiklikler genel message ayrıştırma algoritmasını değiştirmemekle birlikte message anlambilimine eklemeler yapabilecek ve gönderenin ek yeteneklerini ima edebilecek özellikler eklediğinde artırılır. <major> numarası, protokol içindeki bir message'ın formatı değiştiğinde artırılır.

Bir HTTP message'ının versiyonu, message'ın ilk satırındaki HTTP-Version alanıyla belirtilir. Protokol versiyonu belirtilmemişse alıcı, message'ın basit HTTP/0.9 formatında olduğunu varsaymalıdır.

HTTP-Version = "HTTP" "/" 1DIGIT "." 1DIGIT

Major ve minor numaraların ayrı tam sayılar olarak değerlendirilmesi ve her birinin tek bir basamağın ötesine geçebileceği unutulmamalıdır. Bu nedenle HTTP/2.4, HTTP/2.13'ten düşük bir versiyondur; HTTP/2.13 ise HTTP/12.3'ten düşüktür. Baştaki sıfırlar alıcılar tarafından göz ardı edilmeli ve gönderenler tarafından hiçbir zaman üretilmemelidir.

Bu belge, HTTP protokolünün 0.9 ve 1.0 versiyonlarını tanımlamaktadır. Bu spesifikasyon tarafından tanımlanan Full-Request veya Full-Response message'larını gönderen uygulamaların "HTTP/1.0" değerinde bir HTTP-Version içermesi zorunludur.

HTTP/1.0 server'ları şunları yapmalıdır:

- *HTTP/0.9 ve HTTP/1.0 request'leri için Request-Line formatını tanımalıdır;*
- *HTTP/0.9 veya HTTP/1.0 formatındaki herhangi bir geçerli request'i anlamalıdır;*
- *client tarafından kullanılan protokol versiyonuyla aynı versiyonda bir message ile uygun şekilde yanıt vermelidir.*

HTTP/1.0 client'ları şunları yapmalıdır:

- *HTTP/1.0 response'ları için Status-Line formatını tanımalıdır;*
- *HTTP/0.9 veya HTTP/1.0 formatındaki herhangi bir geçerli response'u anlamalıdır.*

Proxy ve gateway uygulamaları, uygulamanın yerel HTTP versiyonundan farklı bir formatta alınan request'leri iletirken dikkatli olmalıdır. Protokol versiyonu gönderenin protokol kapasitesini belirttiğinden, bir proxy/gateway hiçbir zaman kendi yerel versiyonundan büyük bir versiyon göstergesiyle message göndermemelidir; daha yüksek versiyonlu bir request alınırsa proxy/gateway ya request versiyonunu düşürmelí ya da bir hata ile yanıt vermelidir. Uygulamanın yerel formatından daha düşük versiyonlu request'ler iletilmeden önce yükseltilebilir; proxy/gateway'in o request'e verdiği response, yukarıda listelenen server gereksinimlerini karşılamalıdır.

3.2 Uniform Resource Identifier'lar

URI'lar pek çok adla tanınmıştır: WWW adresleri, Universal Document Identifier'lar, Universal Resource Identifier'lar [2] ve son olarak Uniform Resource Locator'ların (URL) [4] ve Name'lerin (URN) [16] birleşimi. HTTP açısından Uniform Resource Identifier'lar, bir ağ kaynağını ad, konum veya başka herhangi bir özellik aracılığıyla tanımlayan basit biçimlendirilmiş dizelerdir.

3.2.1 Genel Sözdizimi

HTTP'deki URI'lar, kullanım bağlamına bağlı olarak mutlak biçimde veya bilinen bir temel URI'ya [9] göreli olarak ifade edilebilir. İki biçim, mutlak URI'ların her zaman bir scheme adı ve ardından iki nokta üst üste ile başlaması gerçeğiyle birbirinden ayrılır.

*URI = (absoluteURI | relativeURI) ["#" fragment] absoluteURI = scheme ":" *(uchar | reserved) relativeURI = net_path | abs_path | rel_path net_path = "//" net_loc [abs_path] abs_path = "/" rel_path rel_path = [path] [";" params] ["?" query] path = fsegment ("/" segment) fsegment = 1pchar segment = *pchar params = param *(";" param) param = (pchar | "/") scheme = 1(ALPHA | DIGIT | "+" | "-" | ".") net_loc = *(pchar | ";" | "?") query = *(uchar | reserved) fragment = (uchar | reserved) pchar = uchar | ":" | "@" | "&" | "=" | "+" uchar = unreserved | escape unreserved = ALPHA | DIGIT | safe | extra | national escape = "%" HEX HEX reserved = ";" | "/" | "?" | ":" | "@" | "&" | "=" | "+" extra = "!" | "" | "'" | "(" | ")" | "," safe = "$" | "-" | "_" | "." unsafe = CTL | SP | <"> | "#" | "%" | "<" | ">" national = <ALPHA, DIGIT, reserved, extra, safe ve unsafe dışındaki herhangi bir OCTET>*

URL sözdizimi ve anlambilimi hakkında kesin bilgi için bkz. RFC 1738 [4] ve RFC 1808 [9]. Yukarıdaki BNF, RFC 1738 tarafından belirtilen geçerli URL'lerde izin verilmeyen ulusal karakterleri içermektedir; zira HTTP server'ları, adreslerin rel_path bölümünü temsil etmek için izin verilen ayrılmamış karakter setiyle kısıtlanmamıştır ve HTTP proxy'leri RFC 1738 tarafından tanımlanmayan URI'lar için request alabilir.

3.2.2 http URL

"http" scheme'i, HTTP protokolü aracılığıyla ağ kaynaklarını bulmak için kullanılır. Bu bölüm, http URL'leri için scheme'e özgü sözdizimini ve anlambilimini tanımlar.

*http_URL = "http:" "//" host [":" port] [abs_path] host = <RFC 1123'ün 2.1. Bölümünde tanımlandığı üzere geçerli bir İnternet host alan adı veya IP adresi (noktalı ondalık biçimde)> port = *DIGIT*

Port boş veya belirtilmemişse port 80 varsayılır. Anlambilim şu şekildedir: tanımlanan kaynak, o host'un söz konusu portunda TCP connection'larını dinleyen server'da yer almaktadır ve kaynağın Request-URI'si abs_path'tir. abs_path URL'de mevcut değilse Request-URI olarak kullanıldığında "/" olarak verilmelidir (Bölüm 5.1.2).

Not: HTTP protokolü transport katmanı protokolünden bağımsız olsa da http URL yalnızca kaynakları TCP konumlarıyla tanımlar; bu nedenle TCP dışı kaynakların başka bir URI scheme'iyle tanımlanması gerekir.

"http" URL'lerinin kanonik biçimi, host içindeki tüm UPALPHA karakterlerinin LOALPHA karşılıklarına dönüştürülmesiyle (hostname'ler büyük/küçük harf duyarsızdır), port 80 ise [":" port] kısmının çıkarılmasıyla ve boş abs_path yerine "/" kullanılmasıyla elde edilir.

3.3 Tarih/Saat Formatları

HTTP/1.0 uygulamaları, tarih/saat damgalarının gösterimi için tarihsel olarak üç farklı formatı desteklemiştir:

Sun, 06 Nov 1994 08:49:37 GMT ; RFC 822, RFC 1123 ile güncellenmiş Sunday, 06-Nov-94 08:49:37 GMT ; RFC 850, RFC 1036 ile geçersiz kılınmış Sun Nov 6 08:49:37 1994 ; ANSI C'nin asctime() formatı

İlk format, RFC 1123 [6] tarafından tanımlanan sabit uzunluklu bir alt kümeyi temsil eden İnternet standardı olarak tercih edilmektedir (RFC 822 [7]'nin güncellemesi). İkinci format yaygın olarak kullanılmakla birlikte, geçerliliğini yitirmiş RFC 850 [10] tarih formatına dayanmakta ve dört basamaklı bir yıldan yoksundur. Tarih değerini ayrıştıran HTTP/1.0 client ve server'ları üç formatı da kabul etmelidir; ancak hiçbir zaman üçüncü (asctime) formatı üretmemelidir.

Not: Tarih değerlerinin alıcıları, proxy/gateway aracılığıyla SMTP veya NNTP üzerinden message alırken veya gönderirken zaman zaman olduğu gibi, HTTP dışı uygulamalar tarafından üretilmiş olabilecek tarih değerlerini kabul etmede esnek davranmaları için teşvik edilmektedir.

Tüm HTTP/1.0 tarih/saat damgalarının istisnasız olarak Evrensel Zaman (UT), diğer adıyla Greenwich Mean Time (GMT) cinsinden ifade edilmesi zorunludur. İlk iki formatta bu durum saat dilimi için üç harfli "GMT" kısaltmasının kullanılmasıyla belirtilir; asctime formatı okunurken de bu varsayılmalıdır.

HTTP-date = rfc1123-date | rfc850-date | asctime-date rfc1123-date = wkday "," SP date1 SP time SP "GMT" rfc850-date = weekday "," SP date2 SP time SP "GMT" asctime-date = wkday SP date3 SP time SP 4DIGIT date1 = 2DIGIT SP month SP 4DIGIT ; gün ay yıl (örn. 02 Jun 1982) date2 = 2DIGIT "-" month "-" 2DIGIT ; gün-ay-yıl (örn. 02-Jun-82) date3 = month SP (2DIGIT | (SP 1DIGIT)) ; ay gün (örn. Jun 2) time = 2DIGIT ":" 2DIGIT ":" 2DIGIT ; 00:00:00 - 23:59:59 wkday = "Mon" | "Tue" | "Wed" | "Thu" | "Fri" | "Sat" | "Sun" weekday = "Monday" | "Tuesday" | "Wednesday" | "Thursday" | "Friday" | "Saturday" | "Sunday" month = "Jan" | "Feb" | "Mar" | "Apr" | "May" | "Jun" | "Jul" | "Aug" | "Sep" | "Oct" | "Nov" | "Dec"

Not: HTTP'nin tarih/saat damgası formatına ilişkin gereksinimleri yalnızca protokol akışı içindeki kullanım için geçerlidir. Client ve server'ların kullanıcıya sunum, request günlüğü tutma vb. amaçlarla bu formatları kullanması zorunlu değildir.

3.4 Karakter Setleri

HTTP, "karakter seti" teriminin MIME için tanımlandığı şekliyle aynı tanımını kullanır:

"Karakter seti" terimi bu belgede, bir oktet dizisini bir karakter dizisine dönüştürmek için bir veya daha fazla tabloyla kullanılan yöntemi ifade eder. Ters yönde koşulsuz dönüşümün zorunlu olmadığını; belirli bir karakter setinde tüm karakterlerin bulunmayabileceğini ve bir

karakter setinin belirli bir karakteri temsil etmek için birden fazla oktet dizisi sağlayabileceğini unutmayın. Bu tanım, US-ASCII gibi basit tek tablolu eşlemelerden ISO 2022'nin tekniklerini kullananlar gibi karmaşık tablo değiştirme yöntemlerine kadar çeşitli karakter kodlama türlerine olanak tanımayı amaçlamaktadır. Ancak bir MIME karakter seti adıyla ilişkilendirilen tanımın, oktetlerden karakterlere gerçekleştirilecek eşlemeyi tam olarak belirtmesi zorunludur. Özellikle, kesin eşlemeyi belirlemek amacıyla harici profil bilgisinin kullanılmasına izin verilmez.

Not: "Karakter seti" teriminin bu kullanımı daha yaygın olarak "karakter kodlaması" olarak anılmaktadır. Ancak HTTP ve MIME aynı kayıt defterini paylaştığından terminolojinin de paylaşılması önemlidir.

HTTP karakter setleri, büyük/küçük harf duyarsız token'larla tanımlanır. Token'ların tam kümesi, IANA Karakter Seti kayıt defteri [15] tarafından tanımlanmaktadır. Ancak bu kayıt defteri her karakter seti için tek ve tutarlı bir token tanımlamadığından, HTTP entity'leriyle kullanılma olasılığı en yüksek olan karakter setleri için tercih edilen adlar burada tanımlanmaktadır. Bu karakter setleri, RFC 1521 [5] tarafından kaydedilenleri, yani US-ASCII [17] ve ISO-8859 [18] karakter setlerini ve MIME charset parametreleri içinde kullanım için özel olarak önerilen diğer adları kapsar.

charset = "US-ASCII" | "ISO-8859-1" | "ISO-8859-2" | "ISO-8859-3" | "ISO-8859-4" | "ISO-8859-5" | "ISO-8859-6" | "ISO-8859-7" | "ISO-8859-8" | "ISO-8859-9" | "ISO-2022-JP" | "ISO-2022-JP-2" | "ISO-2022-KR" | "UNICODE-1-1" | "UNICODE-1-1-UTF-7" | "UNICODE-1-1-UTF-8" | token

HTTP, charset değeri olarak keyfi bir token kullanılmasına izin verse de IANA Karakter Seti kayıt defterinde [15] önceden tanımlanmış bir değere sahip herhangi bir token, o kayıt defteri tarafından tanımlanan karakter setini temsil etmelidir. Uygulamalar, karakter seti kullanımlarını IANA kayıt defteri tarafından tanımlananlarla sınırlandırmalıdır.

Bir entity body'nin karakter seti, US-ASCII veya ISO-8859-1 etiketleri yerine etiket kullanılmaması tercih edilmesi hariç, bu body içinde kullanılan karakter kodlarının en küçük ortak paydası olarak etiketlenmelidir.

3.5 Content Coding'ler

Content coding değerleri, bir kaynağa uygulanmış bir kodlama dönüşümünü belirtmek için kullanılır. Content coding'ler öncelikle bir belgenin, temel media type kimliğini kaybetmeden sıkıştırılmasına veya şifrelenmesine olanak tanımak amacıyla kullanılır. Tipik olarak kaynak bu kodlamayla saklanır ve yalnızca işleme tabi tutulmadan önce çözümlenir.

content-coding = "x-gzip" | "x-compress" | token

Not: Gelecekteki uyumluluk için HTTP/1.0 uygulamaları, "gzip" ve "compress"i sırasıyla "x-gzip" ve "x-compress" ile eşdeğer kabul etmelidir.

Tüm content-coding değerleri büyük/küçük harf duyarsızdır. HTTP/1.0, Content-Encoding (Bölüm 10.3) header field'ında content-coding değerlerini kullanır. Değer content-coding'i tanımlasa da daha önemli olan, kodlamayı kaldırmak için hangi çözümleme mekanizmasının

gerekli olduğunu belirtmesidir. Tek bir programın birden fazla content-coding formatını çözümleyebileceğini unutmayın. Bu spesifikasyon tarafından iki değer tanımlanmıştır:

x-gzip Jean-loup Gailly tarafından geliştirilen "gzip" (GNU zip) dosya sıkıştırma programı tarafından üretilen kodlama formatı. Bu format genellikle 32-bit CRC ile birlikte Lempel-Ziv kodlaması (LZ77) kullanır.

x-compress "compress" dosya sıkıştırma programı tarafından üretilen kodlama formatı. Bu format, uyarlamalı Lempel-Ziv-Welch kodlamasıdır (LZW).

Not: Kodlama formatlarını tanımlamak için program adlarının kullanılması tercih edilmez ve gelecekteki kodlamalar için bu uygulamadan kaçınılması gerekir. Buradaki kullanım, iyi bir tasarım değil tarihsel bir alışkanlığı yansıtmaktadır.

3.6 Media Type'lar

HTTP, açık ve genişletilebilir veri türlendirmesi sağlamak amacıyla Content-Type header field'ında (Bölüm 10.5) İnternet Media Type'larını [13] kullanır.

*media-type = type "/" subtype *(";" parameter) type = token subtype = token*

Parametreler, type/subtype'ın ardından özellik/değer çiftleri biçiminde gelebilir.

parameter = attribute "=" value attribute = token value = token | quoted-string

Type, subtype ve parametre özellik adları büyük/küçük harf duyarsızdır. Parametre değerleri, parametre adının anlambilimine bağlı olarak büyük/küçük harfe duyarlı olabilir ya da olmayabilir. Type ile subtype arasında ve bir özellik ile değeri arasında LWS üretilmemelidir. Tanınmayan bir parametre içeren bir media type alındığında, user agent söz konusu media type'ı tanınmayan parametre ve değeri yokmuş gibi değerlendirmelidir.

Bazı eski HTTP uygulamaları media type parametrelerini tanımaz. HTTP/1.0 uygulamaları media type parametrelerini yalnızca bir message'ın içeriğini tanımlamak için zorunlu oldukları durumlarda kullanmalıdır.

Media type değerleri, Internet Assigned Number Authority'ye (IANA [15]) kaydedilir. Media type kayıt süreci RFC 1590 [13]'te özetlenmiştir. Kayıtlı olmayan media type'ların kullanılması önerilmez.

3.6.1 Kanonikalleştirme ve Metin Varsayılanları

İnternet media type'ları kanonik bir biçimle kaydedilir. Genel olarak, HTTP aracılığıyla aktarılan bir Entity-Body, iletilmeden önce uygun kanonik biçimde temsil edilmelidir. Body bir Content-Encoding ile kodlanmışsa temel verinin kodlanmadan önce kanonik biçimde olması gerekir.

"text" type'ının media subtype'ları, kanonik biçimde metin satır sonu olarak CRLF kullanır. Ancak HTTP, Entity-Body içinde tutarlı biçimde kullanıldığında yalnızca CR veya LF'nin satır sonunu temsil ettiği metin medyasının aktarılmasına olanak tanır. HTTP uygulamaları, HTTP

üzerinden alınan metin medyasında satır sonunu temsil eden CRLF, yalnız CR ve yalnız LF'yi kabul etmelidir.

Ayrıca metin medyası, CR ve LF için sırasıyla oktet 13 ve 10'u kullanmayan bir karakter setinde temsil ediliyorsa (bazı çok baytlı karakter setlerinde olduğu gibi) HTTP, söz konusu karakter seti tarafından satır sonları için CR ve LF karşılığı olarak tanımlanan oktet dizilerinin kullanımına izin verir. Satır sonlarına ilişkin bu esneklik yalnızca Entity-Body'deki metin medyası için geçerlidir; HTTP kontrol yapıları (header field'ları ve multipart sınırları gibi) içinde yalnız CR veya LF, CRLF'nin yerine kullanılmamalıdır.

"charset" parametresi, bazı media type'larla birlikte verinin karakter setini (Bölüm 3.4) tanımlamak için kullanılır. Gönderici tarafından açık bir charset parametresi sağlanmadığında, HTTP üzerinden alınan "text" type'ının media subtype'larının varsayılan charset değeri "ISO-8859-1" olarak tanımlanmıştır. "ISO-8859-1" veya alt kümelerinden farklı karakter setlerindeki verinin alıcı tarafından tutarlı biçimde yorumlanabilmesi için uygun bir charset değeriyle etiketlenmesi zorunludur.

Not: Pek çok mevcut HTTP server'ı uygun etiketleme yapmaksızın "ISO-8859-1" dışındaki karakter setlerini kullanarak veri sunmaktadır. Bu durum birlikte çalışabilirliği azaltmakta ve önerilmemektedir. Bu durumu telafi etmek için bazı HTTP user agent'ları, charset parametresi verilmediğinde media type karakter setinin varsayılan yorumunu kullanıcının değiştirebilmesine olanak tanıyan bir yapılandırma seçeneği sunar.

3.6.2 Multipart Type'lar

MIME, tek bir message'ın Entity-Body içinde birden fazla entity'nin kapsüllenmesine yönelik çeşitli "multipart" type'lar sunar. IANA [15] tarafından kaydedilen multipart type'ların HTTP/1.0 için özel bir anlamı yoktur; ancak user agent'ların her body parçasının amacını doğru yorumlayabilmek için her type'ı anlaması gerekebilir. Bir HTTP user agent'ı, bir multipart type alındığında MIME user agent'ının davranışına aynı ya da benzer şekilde uymalıdır. HTTP server'ları tüm HTTP client'larının multipart type'ları işlemeye hazır olduğunu varsaymamalıdır.

Tüm multipart type'lar ortak bir sözdizimini paylaşır ve media type değerinin bir parçası olarak bir boundary parametresi içermelidir. Message body'nin kendisi bir protokol öğesidir ve bu nedenle body parçaları arasındaki satır sonlarını temsil etmek için yalnızca CRLF kullanmalıdır. Multipart body parçaları, o parçanın anlamı açısından önem taşıyan HTTP header field'larını içerebilir.

3.7 Ürün Token'ları

Ürün token'ları, iletişim halindeki uygulamaların kendilerini isteğe bağlı bir eğik çizgi ve versiyon belirleyicisiyle birlikte basit bir ürün token'ı aracılığıyla tanımlamasına olanak tanır. Ürün token'ı kullanan çoğu alan, uygulamanın önemli bir bölümünü oluşturan ve boşlukla ayrılan alt ürünlere de izin verir. Gelenek olarak ürünler, uygulamayı tanımlamadaki önem derecelerine göre sıralanır.

product = token ["/" product-version] product-version = token

Örnekler: User-Agent: CERN-LineMode/2.15 libwww/2.17b3 Server: Apache/0.8.4

Ürün token'ları kısa ve öz olmalıdır; reklam veya diğer ek bilgiler için kullanılmaları açıkça yasaktır. Bir product-version içinde herhangi bir token karakteri yer alabilse de bu token yalnızca bir versiyon tanımlayıcısı olarak kullanılmalıdır (yani aynı ürünün ardışık versiyonları yalnızca ürün değerinin product-version kısmında farklılık göstermelidir).

4. HTTP Message

4.1 Message Type'ları

HTTP message'ları, client'tan server'a request'lerden ve server'dan client'a response'lardan oluşur.

HTTP-message = Simple-Request ; HTTP/0.9 message'ları | Simple-Response | Full-Request ; HTTP/1.0 message'ları | Full-Response

Full-Request ve Full-Response, entity aktarmak için RFC 822 [7]'nin genel message formatını kullanır. Her iki message da isteğe bağlı header field'ları (diğer adıyla "header'lar") ve bir entity body içerebilir. Entity body, header'lardan boş bir satırla (yani yalnızca CRLF içeren bir satırla) ayrılır.

*Full-Request = Request-Line ; Bölüm 5.1 *(General-Header ; Bölüm 4.3 | Request-Header ; Bölüm 5.2 | Entity-Header) ; Bölüm 7.1 CRLF [Entity-Body] ; Bölüm 7.2*

*Full-Response = Status-Line ; Bölüm 6.1 *(General-Header ; Bölüm 4.3 | Response-Header ; Bölüm 6.2 | Entity-Header) ; Bölüm 7.1 CRLF [Entity-Body] ; Bölüm 7.2*

Simple-Request ve Simple-Response herhangi bir header bilgisinin kullanılmasına izin vermez ve tek bir request method'uyla (GET) sınırlıdır.

Simple-Request = "GET" SP Request-URI CRLF Simple-Response = [Entity-Body]

Simple-Request formatının kullanımı önerilmez; zira server'ın döndürülen entity'nin media type'ını belirlemesini engeller.

4.2 Message Header'ları

General-Header (Bölüm 4.3), Request-Header (Bölüm 5.2), Response-Header (Bölüm 6.2) ve Entity-Header (Bölüm 7.1) field'larını kapsayan HTTP header field'ları, RFC 822 [7]'nin 3.1. Bölümünde verilen genel formatla aynı formatı izler. Her header field'ı, hemen ardından iki nokta üst üste (":"), tek bir boşluk (SP) karakteri ve field değeri gelen bir addan oluşur. Field adları büyük/küçük harf duyarsızdır. Header field'ları, her ek satırın başına en az bir SP veya HT eklenerek birden fazla satıra genişletilebilir; ancak bu önerilmez.

*HTTP-header = field-name ":" [field-value] CRLF field-name = token field-value = *(field-content | LWS) field-content = <field-value'yu oluşturan ve *TEXT ya da token, tspecials ve quoted-string kombinasyonlarından oluşan OCTET'ler>*

Header field'larının alınma sırası önem taşımaz. Ancak önce General-Header field'larını, ardından Entity-Header field'larından önce Request-Header veya Response-Header field'larını göndermek "iyi bir uygulama"dır.

Aynı field adına sahip birden fazla HTTP header field'ı, o header field'ının tüm field değeri virgülle ayrılmış bir liste olarak tanımlanmışsa (yani #(değerler)) bir message'da yer alabilir. Birden fazla header field'ının, mesajın anlambilimini değiştirmeksizin her sonraki field-value değeri bir virgülle ayrılarak ilkine eklenmek suretiyle tek bir "field-name: field-value" çiftinde birleştirilmesi mümkün olmalıdır.

4.3 Genel Header Field'ları

Hem request hem de response message'ları için genel geçerliliği bulunan, ancak aktarılan entity için değil yalnızca iletilen message için geçerli olan birkaç header field'ı mevcuttur.

General-Header = Date ; Bölüm 10.6 | Pragma ; Bölüm 10.12

Genel header field adları yalnızca protokol versiyonundaki bir değişiklikle birlikte güvenilir biçimde genişletilebilir. Ancak yeni veya deneysel header field'larına, iletişimdeki tüm taraflar bunları genel header field'ları olarak tanıdığı sürece genel header field'larının anlambilimi verilebilir. Tanınmayan header field'ları Entity-Header field'ları olarak değerlendirilir.

5. *Request*

Client'tan server'a gönderilen bir request message'ı, o message'ın ilk satırında uygulanacak method'u, kaynağın tanımlayıcısını ve kullanılan protokol versiyonunu içerir. Daha sınırlı HTTP/0.9 protokolüyle geriye dönük uyumluluk için bir HTTP request'inin iki geçerli formatı bulunur:

Request = Simple-Request | Full-Request

Simple-Request = "GET" SP Request-URI CRLF

*Full-Request = Request-Line ; Bölüm 5.1 *(General-Header ; Bölüm 4.3 | Request-Header ; Bölüm 5.2 | Entity-Header) ; Bölüm 7.1 CRLF [Entity-Body] ; Bölüm 7.2*

Bir HTTP/1.0 server'ı Simple-Request aldığında HTTP/0.9 Simple-Response ile yanıt vermelidir. Full-Response alabilen HTTP/1.0 client'ları hiçbir zaman Simple-Request oluşturmamalıdır.

5.1 Request-Line

Request-Line, bir method token'ıyla başlar; ardından Request-URI ve protokol versiyonu gelir ve CRLF ile sona erer. Öğeler SP karakterleriyle ayrılır. Son CRLF dizisi dışında CR veya LF'ye izin verilmez.

Request-Line = Method SP Request-URI SP HTTP-Version CRLF

Simple-Request ile Full-Request'in Request-Line'ı arasındaki farkın HTTP-Version field'ının varlığı ve GET dışındaki method'ların kullanılabilirliği olduğuna dikkat edilmelidir.

5.1.1 Method

Method token'ı, Request-URI tarafından tanımlanan kaynak üzerinde gerçekleştirilecek method'u belirtir. Method büyük/küçük harfe duyarlıdır.

Method = "GET" ; Bölüm 8.1 | "HEAD" ; Bölüm 8.2 | "POST" ; Bölüm 8.3 | extension-method

extension-method = token

Belirli bir kaynak tarafından kabul edilebilir method'ların listesi dinamik olarak değişebilir; bir method kaynakta kullanılamıyorsa client bunu response'un dönüş kodu aracılığıyla öğrenir. Server'lar, method tanınmıyorsa veya gerçeklenmemişse 501 (gerçeklenmemiş) status code'unu döndürmelidir.

HTTP/1.0 uygulamalarının yaygın olarak kullandığı method'lar Bölüm 8'de ayrıntılı biçimde tanımlanmıştır.

5.1.2 Request-URI

Request-URI, bir Uniform Resource Identifier'dır (Bölüm 3.2) ve request'in uygulanacağı kaynağı tanımlar.

Request-URI = absoluteURI | abs_path

Request-URI için iki seçenek, request'in niteliğine bağlıdır.

absoluteURI biçimine yalnızca request bir proxy'ye yapıldığında izin verilir. Proxy'den request'i iletmesi ve response'u döndürmesi istenir. Request GET veya HEAD ise ve daha önceki bir response önbelleğe alınmışsa proxy, Expires header field'ındaki kısıtlamaları geçerse önbelleğe alınmış message'ı kullanabilir. Proxy'nin request'i başka bir proxy'ye veya absoluteURI'nin belirlediği server'a doğrudan iletebileceğine dikkat edilmelidir. Request döngülerini önlemek amacıyla proxy, tüm takma adlar, yerel varyasyonlar ve sayısal IP adresi dahil kendi server adlarının tamamını tanıyabilmelidir. Örnek bir Request-Line şu şekildedir:

GET http://www.w3.org/pub/WWW/TheProject.html HTTP/1.0

Request-URI'nin en yaygın biçimi, bir origin server veya gateway üzerindeki bir kaynağı tanımlamak için kullanılanıdır. Bu durumda yalnızca URI'nin mutlak yolu iletilir (bkz. Bölüm 3.2.1, abs_path). Örneğin yukarıdaki kaynağı doğrudan origin server'dan almak isteyen bir client, "www.w3.org" host'unun 80 numaralı portuna TCP bağlantısı kurarak şu satırı gönderir:

GET /pub/WWW/TheProject.html HTTP/1.0

ardından Full-Request'in geri kalanını ekler. Mutlak yolun boş olamayacağına dikkat edilmelidir; özgün URI'de yoksa "/" (server kökü) olarak verilmesi zorunludur.

Request-URI, RFC 1738 [4] tarafından tanımlanan "% HEX HEX" kodlaması kullanılarak bazı karakterlerin escape edildiği bir kodlanmış dize olarak iletilir. Origin server, request'i doğru biçimde yorumlayabilmek için Request-URI'yi çözümlemek zorundadır.

5.2 Request Header Field'ları

Request header field'ları, client'ın request hakkında ve kendisi hakkında sunucuya ek bilgi iletmesine olanak tanır. Bu field'lar, bir programlama dili method (yordam) çağrısındaki parametrelerle eşdeğer anlambilimle request modifier'ları olarak işlev görür.

Request-Header = Authorization ; Bölüm 10.2 | From ; Bölüm 10.8 | If-Modified-Since ; Bölüm 10.9 | Referer ; Bölüm 10.13 | User-Agent ; Bölüm 10.15

Request-Header field adları yalnızca protokol versiyonundaki bir değişiklikle birlikte güvenilir biçimde genişletilebilir. Ancak yeni veya deneysel header field'larına, iletişimdeki tüm taraflar bunları request header field'ları olarak tanıdığı sürece request header field'larının anlambilimi verilebilir. Tanınmayan header field'ları Entity-Header field'ları olarak değerlendirilir.

6. *Response*

Bir request message'ını aldıktan ve yorumladıktan sonra server, bir HTTP response message'ı biçiminde yanıt verir.

Response = Simple-Response | Full-Response

Simple-Response = [Entity-Body]

*Full-Response = Status-Line ; Bölüm 6.1 *(General-Header ; Bölüm 4.3 | Response-Header ; Bölüm 6.2 | Entity-Header) ; Bölüm 7.1 CRLF [Entity-Body] ; Bölüm 7.2*

Simple-Response yalnızca HTTP/0.9 Simple-Request'ine yanıt olarak veya server yalnızca daha sınırlı HTTP/0.9 protokolünü destekliyorsa gönderilmelidir. Bir client HTTP/1.0 Full-Request gönderir ve Status-Line ile başlamayan bir response alırsa bu response'un Simple-Response olduğunu varsaymalı ve buna göre ayrıştırmalıdır. Simple-Response'un yalnızca entity body'den oluştuğunu ve server'ın connection'ı kapatmasıyla sonlandığını unutmayın.

6.1 Status-Line

Full-Response message'ının ilk satırı Status-Line'dır; protokol versiyonu, ardından sayısal bir status code ve onunla ilişkili metinsel ifade yer alır; her öğe SP karakterleriyle ayrılır. Son CRLF dizisi dışında CR veya LF'ye izin verilmez.

Status-Line = HTTP-Version SP Status-Code SP Reason-Phrase CRLF

Bir status line her zaman protokol versiyonu ve status code ile başladığından:

"HTTP/" 1DIGIT "." 1DIGIT SP 3DIGIT SP

(örn. "HTTP/1.0 200 "), bu ifadenin varlığı Full-Response'u Simple-Response'dan ayırt etmek için yeterlidir.

6.1.1 Status Code ve Reason Phrase

Status-Code öğesi, request'i anlama ve karşılama girişiminin 3 basamaklı bir tam sayı sonuç kodudur. Reason-Phrase, Status-Code'un kısa bir metinsel açıklamasını sunmayı amaçlar. Status-Code otomasyonlar tarafından kullanılmak üzere tasarlanmıştır; Reason-Phrase ise insan kullanıcıya yöneliktir. Client'ın Reason-Phrase'i incelemesi veya görüntülemesi zorunlu değildir.

Status-Code'un ilk basamağı response sınıfını tanımlar. Son iki basamağın herhangi bir sınıflandırma işlevi yoktur. İlk basamak için 5 değer mevcuttur:

1xx: Bilgi amaçlı - Kullanılmıyor, ancak gelecekte kullanım için ayrılmış 2xx: Başarı - Eylem başarıyla alındı, anlaşıldı ve kabul edildi. 3xx: Yönlendirme - Request'i tamamlamak için ek eylem gerekiyor 4xx: Client Hatası - Request hatalı sözdizimi içeriyor veya karşılanamıyor 5xx: Server Hatası - Server görünüşte geçerli bir request'i karşılamayı başaramadı

HTTP/1.0 için tanımlanan sayısal status code'larının bireysel değerleri ve örnek Reason-Phrase'ler aşağıda sunulmuştur. Burada listelenen reason phrase'ler yalnızca önerilmektedir; protokolü etkilemeksizin yerel karşılıklarıyla değiştirilebilirler. Bu kodlar Bölüm 9'da ayrıntılı biçimde tanımlanmıştır.

Status-Code = "200" ; OK | "201" ; Created | "202" ; Accepted | "204" ; No Content | "301" ; Moved Permanently | "302" ; Moved Temporarily | "304" ; Not Modified | "400" ; Bad Request | "401" ; Unauthorized | "403" ; Forbidden | "404" ; Not Found | "500" ; Internal Server Error | "501" ; Not Implemented | "502" ; Bad Gateway | "503" ; Service Unavailable | extension-code

*extension-code = 3DIGIT Reason-Phrase = *<CR, LF hariç TEXT>*

HTTP status code'ları genişletilebilir; ancak yukarıdaki kodlar mevcut pratikte genel olarak tanınan tek kodlardır. HTTP uygulamalarının kayıtlı tüm status code'larının anlamını kavraması zorunlu değildir; ancak bu elbette tercih edilen bir durumdur. Bununla birlikte uygulamalar, ilk basamağın gösterdiği şekilde herhangi bir status code'unun sınıfını anlamalı ve tanımadıkları bir response'u, tanımadıkları response önbelleğe alınmaması kaydıyla o sınıfın x00 status code'uyla eşdeğer kabul etmelidir. Örneğin client 431 gibi tanımadığı bir status code alırsa request'iyle ilgili bir sorun olduğunu güvenle varsayabilir ve response'u sanki 400 status code'u almış gibi değerlendirebilir. Bu tür durumlarda user agent'lar, response ile birlikte dönen entity'yi kullanıcıya sunmalıdır; zira söz konusu entity, olağandışı durumu açıklayan insan tarafından okunabilir bilgiler içeriyor olabilir.

6.2 Response Header Field'ları

Response header field'ları, server'ın Status-Line içine yerleştirilemeyen response hakkındaki ek bilgileri iletmesine olanak tanır. Bu header field'ları, server hakkında ve Request-URI tarafından tanımlanan kaynağa gelecekteki erişim hakkında bilgi sağlar.

Response-Header = Location ; Bölüm 10.11 | Server ; Bölüm 10.14 | WWW-Authenticate ; Bölüm 10.16

Response-Header field adları yalnızca protokol versiyonundaki bir değişiklikle birlikte güvenilir biçimde genişletilebilir. Ancak yeni veya deneysel header field'larına, iletişimdeki tüm taraflar bunları response header field'ları olarak tanıdığı sürece response header field'larının anlambilimi verilebilir. Tanınmayan header field'ları Entity-Header field'ları olarak değerlendirilir.

7. *Entity*

Full-Request ve Full-Response message'ları bazı request ve response'larda bir entity aktarabilir. Bir entity, Entity-Header field'larından ve (genellikle) bir Entity-Body'den oluşur. Bu bölümde gönderici ve alıcı, entity'yi kimin gönderip kimin aldığına bağlı olarak client veya server'a atıfta bulunur.

7.1 Entity Header Field'ları

Entity-Header field'ları, Entity-Body hakkındaki isteğe bağlı metabilgileri ya da body mevcut değilse request tarafından tanımlanan kaynak hakkındaki bilgileri tanımlar.

Entity-Header = Allow ; Bölüm 10.1 | Content-Encoding ; Bölüm 10.3 | Content-Length ; Bölüm 10.4 | Content-Type ; Bölüm 10.5 | Expires ; Bölüm 10.7 | Last-Modified ; Bölüm 10.10 | extension-header

extension-header = HTTP-header

Extension-header mekanizması, protokolü değiştirmeksizin ek Entity-Header field'larının tanımlanmasına olanak tanır; ancak bu field'ların alıcı tarafından tanınacağı varsayılamaz. Tanınmayan header field'ları alıcı tarafından göz ardı edilmeli ve proxy'ler tarafından iletilmelidir.

7.2 Entity Body

Bir HTTP request veya response ile birlikte gönderilen entity body (varsa), Entity-Header field'ları tarafından tanımlanan bir format ve kodlamadadır.

*Entity-Body = *OCTET*

Bir entity body, request message'ına yalnızca request method bunu gerektirdiğinde dahil edilir. Bir request'te entity body'nin varlığı, request message header'larına Content-Length header field'ının eklenmesiyle bildirilir. Entity body içeren HTTP/1.0 request'leri geçerli bir Content-Length header field'ı içermelidir.

Response message'ları için bir message'a entity body'nin eklenip eklenmeyeceği hem request method'una hem de response code'una bağlıdır. HEAD request method'una verilen tüm response'lar, entity header field'larının varlığı aksini düşündürse bile body içermemelidir. Tüm 1xx (bilgi amaçlı), 204 (içerik yok) ve 304 (değiştirilmemiş) response'ları body içermemelidir. Diğer tüm response'lar bir entity body ya da sıfır (0) değeriyle tanımlı bir Content-Length header field'ı içermelidir.

7.2.1 Type

Bir message'a Entity-Body dahil edildiğinde, body'nin veri type'ı Content-Type ve Content-Encoding header field'ları aracılığıyla belirlenir. Bunlar iki katmanlı, sıralı bir kodlama modeli tanımlar:

entity-body := Content-Encoding(Content-Type(data))

Content-Type, temel verinin media type'ını belirtir. Content-Encoding, type'a uygulanan ve genellikle veri sıkıştırma amacıyla kullanılan ek content coding'i belirtmek için kullanılabilir; bu, istenen kaynağın bir özelliğidir. Content encoding için varsayılan değer yoktur (yani kimlik fonksiyonu).

Entity body içeren herhangi bir HTTP/1.0 message'ı, o body'nin media type'ını tanımlayan bir Content-Type header field'ı içermelidir. Media type yalnızca Content-Type header tarafından verilmemişse (Simple-Response message'larında olduğu gibi), alıcı içeriğini ve/veya kaynağı tanımlamak için kullanılan URL'nin adı uzantısını inceleyerek media type'ı tahmin etmeye çalışabilir. Media type bilinmiyorsa alıcı onu "application/octet-stream" type'ı olarak değerlendirmelidir.

7.2.2 Length

Bir message'a Entity-Body dahil edildiğinde, body'nin uzunluğu iki yoldan biriyle belirlenebilir. Content-Length header field'ı mevcutsa değeri, Entity-Body'nin bayt cinsinden uzunluğunu temsil eder. Aksi takdirde body uzunluğu, server'ın connection'ı kapatmasıyla belirlenir.

Request body'sinin sonunu belirtmek için connection'ın kapatılması kullanılamaz; zira bu durumda server'ın response göndermesi için herhangi bir olanak kalmaz. Bu nedenle entity body içeren HTTP/1.0 request'leri geçerli bir Content-Length header field'ı içermelidir. Bir request entity body içeriyor ve Content-Length belirtilmemişse ve server uzunluğu diğer field'lardan tanıyamıyor veya hesaplayamıyorsa server 400 (hatalı request) response'u göndermelidir.

Not: Bazı eski server'lar, dinamik olarak veri akışına eklenen server-side include'lar içeren bir belge gönderirken geçersiz Content-Length değerleri sağlamaktadır. Bunun HTTP'nin gelecekteki versiyonları tarafından tolere edilmeyeceği vurgulanmalıdır. Client, yanıtı uyumlu bir server'dan aldığını bilmediği sürece Content-Length değerinin doğruluğuna güvenmemelidir.

8. Method Tanımları

HTTP/1.0 için yaygın method'ların kümesi aşağıda tanımlanmıştır. Bu küme genişletilebilse de ek method'ların ayrı ayrı genişletilmiş client ve server'lar için aynı anlambilimi paylaşacağı varsayılamaz.

8.1 GET

GET method'u, Request-URI tarafından tanımlanan herhangi bir bilgiyi (entity biçiminde) alma anlamına gelir. Request-URI bir veri üreten sürece atıfta bulunuyorsa, sürecin kaynak

metni değil, üretilen veri response'ta entity olarak döndürülmelidir; ancak bu metin sürecin çıktısıysa durum farklıdır.

GET method'unun anlambilimi, request message bir If-Modified-Since header field'ı içeriyorsa "koşullu GET" olarak değişir. Koşullu GET method'u, tanımlanan kaynağın yalnızca If-Modified-Since header'ında belirtilen tarihten bu yana değiştirilmişse aktarılmasını talep eder; bu durum Bölüm 10.9'da açıklanmıştır. Koşullu GET method'u, birden fazla request gerektirmeksizin veya gereksiz veri aktarmaksızın önbelleğe alınmış entity'lerin yenilenmesine olanak tanıyarak ağ kullanımını azaltmayı amaçlar.

8.2 HEAD

HEAD method'u, server'ın response'ta herhangi bir Entity-Body döndürmemesi gerektiği dışında GET ile aynıdır. HEAD request'ine response olarak HTTP header'larındaki metabilgi, GET request'ine yanıt olarak gönderilen bilgiyle aynı olmalıdır. Bu method, Entity-Body'nin kendisini aktarmaksızın Request-URI tarafından tanımlanan kaynak hakkında metabilgi edinmek için kullanılabilir. Bu method çoğunlukla hyper link'lerin geçerliliğini, erişilebilirliğini ve son değiştirilme tarihini test etmek için kullanılır.

Koşullu GET'e benzer bir "koşullu HEAD" request'i yoktur. HEAD request'ine If-Modified-Since header field'ı eklenirse bu göz ardı edilmelidir.

8.3 POST

POST method'u, hedef server'ın request'te yer alan entity'yi, Request-Line içindeki Request-URI tarafından tanımlanan kaynağın yeni bir alt öğesi olarak kabul etmesini talep etmek için kullanılır. POST, aşağıdaki işlevleri tek tip bir method olarak kapsayacak şekilde tasarlanmıştır:

- *Mevcut kaynaklara açıklama ekleme;*
- *Bir bülten panosuna, haber grubuna, posta listesine veya benzer bir makale grubuna mesaj gönderme;*
- *Form gönderme sonucu [3] gibi bir veri bloğunu veri işleme sürecine sağlama;*
- *Bir veritabanını ekleme işlemi aracılığıyla genişletme.*

POST method'u tarafından gerçekleştirilen asıl işlev server tarafından belirlenir ve genellikle Request-URI'ye bağlıdır. Gönderilen entity, bir dosyanın onu içeren dizine, bir haber makalesinin gönderildiği haber grubuna veya bir kaydın veritabanına bağımlı olduğu şekilde o URI'ye bağımlıdır.

Başarılı bir POST işlemi, entity'nin origin server üzerinde bir kaynak olarak oluşturulmasını veya gelecekteki başvuru için erişilebilir kılınmasını zorunlu kılmaz. Yani POST method'u tarafından gerçekleştirilen eylem, bir URI ile tanımlanabilen bir kaynakla sonuçlanmayabilir. Bu durumda, response'un sonucu açıklayan bir entity içerip içermediğine bağlı olarak uygun response status'u 200 (tamam) veya 204 (içerik yok) olacaktır.

Origin server üzerinde bir kaynak oluşturulmuşsa response 201 (oluşturuldu) olmalı ve request'in status'unu açıklayan ve yeni kaynağa başvuran (tercihen "text/html" type'ında) bir entity içermelidir.

Tüm HTTP/1.0 POST request'lerinde geçerli bir Content-Length zorunludur. Bir HTTP/1.0 server'ı, request message'ının içeriğinin uzunluğunu belirleyemiyorsa 400 (hatalı request) message'ı ile yanıt vermelidir.

Uygulamalar POST request'ine verilen response'ları önbelleğe almamalıdır; zira uygulama, server'ın ilerideki bir request'e eşdeğer bir response döndüreceğini bilmenin herhangi bir yoluna sahip değildir.

9. *Status Code Tanımları*

Aşağıda her Status-Code, hangi method'ların ardından gelebileceğine ve response'da gereken metabilgilere ilişkin açıklamalarla birlikte tanımlanmıştır.

9.1 Bilgi Amaçlı 1xx

Bu status code sınıfı, yalnızca Status-Line ve isteğe bağlı header'lardan oluşan ve boş bir satırla sonlanan geçici bir response'u ifade eder. HTTP/1.0, herhangi bir 1xx status code tanımlamaz ve bunlar HTTP/1.0 request'ine geçerli bir response değildir. Ancak bu spesifikasyonun kapsamı dışındaki deneysel uygulamalar için yararlı olabilirler.

9.2 Başarılı 2xx

Bu status code sınıfı, client'ın request'inin başarıyla alındığını, anlaşıldığını ve kabul edildiğini belirtir.

200 OK Request başarılı oldu. Response ile birlikte döndürülen bilgi, request'te kullanılan method'a göre değişir: GET — request edilen kaynağa karşılık gelen entity response'ta gönderilir; HEAD — response yalnızca header bilgisini içermeli, Entity-Body içermemelidir; POST — eylemin sonucunu açıklayan veya içeren bir entity.

201 Created Request yerine getirildi ve yeni bir kaynak oluşturulmasıyla sonuçlandı. Yeni oluşturulan kaynağa, response'un entity'sinde döndürülen URI'ler aracılığıyla başvurulabilir. Origin server, bu Status-Code'u kullanmadan önce kaynağı oluşturmalıdır. Eylem hemen gerçekleştirilemiyorsa server, response body'sine kaynağın ne zaman kullanılabilir olacağına ilişkin bir açıklama eklemeli; aksi takdirde 202 (kabul edildi) ile yanıt vermelidir.

Bu spesifikasyon tarafından tanımlanan method'lar arasında yalnızca POST bir kaynak oluşturabilir.

202 Accepted Request işlenmek üzere kabul edildi; ancak işlem henüz tamamlanmadı. İşlem gerçekten başladığında izin verilmeyebileceğinden request'in sonunda yerine getirilip getirilmeyeceği belirsizdir. Bu tür asenkron işlemler için yeniden status code gönderme imkânı bulunmamaktadır.

202 response kasıtlı olarak bağlayıcı değildir. Amacı, bir server'ın başka bir işlem için (belki de günde yalnızca bir kez çalışan toplu iş gibi) bir request'i kabul etmesine ve bu süreç tamamlanana kadar user agent'ın server bağlantısını sürdürmek zorunda kalmamasına olanak tanımaktır. Bu response ile birlikte döndürülen entity, request'in mevcut durumunu ve ya bir durum izleme göstergesine ya da kullanıcının request'in ne zaman karşılanacağına dair bir tahmine yönlendiren bir bağlantı içermelidir.

204 No Content Server request'i yerine getirdi; ancak geri gönderilecek yeni bir bilgi bulunmuyor. Client bir user agent ise request'i tetikleyen belgeden farklı bir belge görünümüne geçmemelidir. Bu response öncelikle script'lerin veya diğer eylemlerin, user agent'ın etkin belge görünümünde değişikliğe yol açmaksızın gerçekleşmesine olanak tanımak için tasarlanmıştır. Response, entity header'ları biçiminde yeni metabilgi içerebilir; bu bilgiler, user agent'ın aktif görünümünde bulunan belge için geçerli olmalıdır.

9.3 Yönlendirme 3xx

Bu status code sınıfı, request'i yerine getirmek için user agent'ın ek eylem gerçekleştirmesi gerektiğini belirtir. Gerekli eylem, sonraki request'te kullanılan method GET veya HEAD ise kullanıcıyla etkileşim olmaksızın user agent tarafından gerçekleştirilebilir. Bir user agent hiçbir zaman bir request'i 5 defadan fazla otomatik olarak yönlendirmemelidir; bu tür yönlendirmeler genellikle sonsuz bir döngüye işaret eder.

300 Multiple Choices Bu response kodu doğrudan HTTP/1.0 uygulamaları tarafından kullanılmaz; ancak 3xx sınıfı response'ların yorumlanmasında varsayılan olarak hizmet eder.

Request edilen kaynak bir veya daha fazla konumda mevcuttur. HEAD request'i olmadığı sürece response, kullanıcının veya user agent'ın en uygununu seçebileceği kaynak özellikleri ve konumlarından oluşan bir liste içeren bir entity barındırmalıdır. Server'ın tercih ettiği bir seçenek varsa bunu Location field'ına eklemeli; user agent'lar bu field değerini otomatik yönlendirme için kullanabilir.

301 Moved Permanently Request edilen kaynağa yeni kalıcı bir URL atandı ve bu kaynağa gelecekteki tüm başvurular bu URL kullanılarak yapılmalıdır. Bağlantı düzenleme özelliklerine sahip client'lar, mümkün olduğunda Request-URI'ye yapılan referansları server tarafından döndürülen yeni referansla otomatik olarak güncellemelidir.

Yeni URL, response'taki Location field'ı tarafından belirtilmelidir. HEAD request'i olmadığı sürece response'un Entity-Body'si yeni URL'ye bir köprü içeren kısa bir not içermelidir.

POST method'u kullanılarak yapılan bir request'e yanıt olarak 301 status code'u alınırsa user agent, bu durum kullanıcı tarafından onaylanmadığı sürece request'i otomatik olarak yönlendirmemelidir; zira bu, request'in gönderildiği koşulları değiştirebilir.

Not: Bir POST request'ini 301 status code'u alındıktan sonra otomatik olarak yönlendirirken bazı mevcut user agent'lar hatalı biçimde bunu GET request'ine dönüştürmektedir.

302 Moved Temporarily Request edilen kaynak geçici olarak farklı bir URL altında bulunmaktadır. Yönlendirme zaman zaman değişebileceğinden client gelecekteki request'ler için Request-URI'yi kullanmaya devam etmelidir.

URL, response'taki Location field'ı tarafından belirtilmelidir. HEAD request'i olmadığı sürece response'un Entity-Body'si yeni URI'ye bir köprü içeren kısa bir not içermelidir.

POST method'u kullanılarak yapılan bir request'e yanıt olarak 302 status code'u alınırsa user agent, bu durum kullanıcı tarafından onaylanmadığı sürece request'i otomatik olarak yönlendirmemelidir; zira bu, request'in gönderildiği koşulları değiştirebilir.

Not: Bir POST request'ini 302 status code'u alındıktan sonra otomatik olarak yönlendirirken bazı mevcut user agent'lar hatalı biçimde bunu GET request'ine dönüştürmektedir.

304 Not Modified Client koşullu bir GET request'i gerçekleştirdiyse ve erişime izin veriliyorsa ancak belge If-Modified-Since field'ında belirtilen tarih ve saatten bu yana değiştirilmediyse server bu status code ile yanıt vermeli ve client'a Entity-Body göndermemelidir. Response'ta yer alan header field'ları yalnızca cache yöneticileriyle ilgili veya entity'nin Last-Modified tarihinden bağımsız olarak değişmiş olabilecek bilgileri içermelidir. İlgili header field'larına örnekler şunlardır: Date, Server ve Expires. Bir cache, 304 response'unda verilen yeni field değerlerini yansıtmak için önbelleğe alınmış entity'yi güncellemelidir.

9.4 Client Hatası 4xx

4xx status code sınıfı, client'ın hata yapmış göründüğü durumlar için tasarlanmıştır. Client, 4xx kodu alındığında request'i henüz tamamlamamışsa server'a veri göndermeyi derhal durdurmalıdır. HEAD request'ine yanıt verilen durumlar dışında server, hata durumunu ve bunun geçici mi yoksa kalıcı mı olduğunu açıklayan bir entity içermelidir. Bu status code'lar herhangi bir request method'una uygulanabilir.

Not: Client TCP üzerinden veri gönderiyorsa, TCP üzerindeki server gerçeklemeleri input connection'ı kapatmadan önce client'ın response'u içeren paket(ler)in alındığını onaylamasına dikkat etmelidir. Client kapatmanın ardından server'a veri göndermeye devam ederse server'ın denetleyicisi client'a bir sıfırlama paketi gönderir; bu durum, client'ın onaylanmamış input tamponlarını HTTP uygulaması tarafından okunup yorumlanmadan önce silebilir.

400 Bad Request Request, hatalı sözdizimi nedeniyle server tarafından anlaşılamadı. Client, değişiklik yapmaksızın aynı request'i tekrarlamamalıdır.

401 Unauthorized Request, kullanıcı kimlik doğrulaması gerektirmektedir. Response, request edilen kaynak için geçerli bir meydan okuma içeren WWW-Authenticate header field'ını (Bölüm 10.16) içermelidir. Client, uygun bir Authorization header field'ıyla (Bölüm 10.2) request'i yineleyebilir. Request zaten Authorization kimlik bilgilerini içeriyorsa 401 response'u bu kimlik bilgileri için yetkilendirmenin reddedildiğini belirtir. 401 response'u öncekiyle aynı meydan okumayı içeriyorsa ve user agent en az bir kez kimlik doğrulamayı denediyse kullanıcıya response'ta verilen entity sunulmalıdır; zira söz konusu entity ilgili tanı bilgilerini içeriyor olabilir. HTTP erişim kimlik doğrulaması Bölüm 11'de açıklanmıştır.

403 Forbidden Server request'i anladı ancak yerine getirmeyi reddediyor. Yetkilendirme yardımcı olmayacak ve request tekrarlanmamalıdır. Request method'u HEAD değilse ve server neden request'in karşılanmadığını kamuoyuna açıklamak istiyorsa reddin nedenini entity body'de açıklamalıdır. Bu status code genellikle server neden request'in reddedildiğini açıklamak istemediğinde veya başka hiçbir response uygun olmadığında kullanılır.

404 Not Found Server, Request-URI ile eşleşen herhangi bir şey bulamadı. Durumun geçici mi yoksa kalıcı mı olduğuna dair herhangi bir bilgi verilmemektedir. Server bu bilgiyi client'a sunmak istemiyorsa bunun yerine 403 (yasak) status code'u kullanılabilir.

9.5 Server Hatası 5xx

"5" rakamıyla başlayan response status code'ları, server'ın hata yaptığının veya request'i yerine getirme kapasitesinden yoksun olduğunun farkında olduğu durumları belirtir. Client, 5xx kodu alındığında request'i henüz tamamlamamışsa server'a veri göndermeyi derhal durdurmalıdır. HEAD request'ine yanıt verilen durumlar dışında server, hata durumunu ve bunun geçici mi yoksa kalıcı mı olduğunu açıklayan bir entity içermelidir. Bu response kodları herhangi bir request method'una uygulanabilir ve zorunlu herhangi bir header field'ı bulunmamaktadır.

500 Internal Server Error Server, request'i yerine getirmesini önleyen beklenmedik bir durumla karşılaştı.

501 Not Implemented Server, request'i yerine getirmek için gereken işlevselliği desteklemiyor. Bu, server'ın request method'unu tanımadığı ve herhangi bir kaynak için bunu destekleyemediği durumlarda uygun response'dur.

502 Bad Gateway Server, gateway veya proxy olarak işlev görürken request'i yerine getirme girişiminde eriştiği yukarı akış server'ından geçersiz bir response aldı.

503 Service Unavailable Server şu anda geçici aşırı yükleme veya bakım nedeniyle request'i işleyemiyor. Bu durumun bir süre sonra ortadan kalkacak geçici bir koşul olduğu ima edilmektedir.

Not: 503 status code'unun varlığı, server'ın aşırı yüklenmesi durumunda bunu kullanmak zorunda olduğunu ima etmez. Bazı server'lar yalnızca connection'ı reddetmeyi tercih edebilir.

10. Header Field Tanımları

Bu bölüm, yaygın olarak kullanılan tüm HTTP/1.0 header field'larının sözdizimini ve anlambilimini tanımlar. Genel ve entity header field'ları için gönderici ve alıcı, message'ı kimin gönderip kimin aldığına bağlı olarak client veya server'a atıfta bulunur.

10.1 Allow

Allow entity-header field'ı, Request-URI tarafından tanımlanan kaynak tarafından desteklenen method'ların kümesini listeler. Bu field'ın amacı, alıcıyı kaynakla ilişkili geçerli method'lar hakkında bilgilendirmektir. Allow header field'ına POST method'u kullanan bir request'te yer verilmez ve bu nedenle POST entity'sinin bir parçası olarak alınırsa göz ardı edilmelidir.

Allow = "Allow" ":" 1#method

Kullanım örneği: Allow: GET, HEAD

Bu field, client'ın diğer method'ları denemesini engelleyemez. Ancak Allow header field değerinin verdiği göstergeler dikkate alınmalıdır. İzin verilen method'ların fiili kümesi, her request sırasında origin server tarafından belirlenir.

Proxy, belirtilen tüm method'ları anlayamasa bile Allow header field'ını değiştirmemelidir; zira user agent'ın origin server ile iletişim kurmak için başka yolları olabilir.

Allow header field'ı, server tarafından hangi method'ların gerçeklendiğini göstermez.

10.2 Authorization

Bir server ile kimliğini doğrulamak isteyen user agent (genellikle, ancak zorunlu olarak değil, 401 response aldıktan sonra) bunu request'e Authorization request-header field'ı ekleyerek yapabilir. Authorization field değeri, request edilen kaynağın realm'i için user agent'ın kimlik doğrulama bilgilerini içeren kimlik bilgilerinden oluşur.

Authorization = "Authorization" ":" credentials

HTTP erişim kimlik doğrulaması Bölüm 11'de açıklanmıştır. Bir request kimliği doğrulanmış ve bir realm belirtilmişse aynı kimlik bilgileri bu realm içindeki diğer tüm request'ler için geçerli olmalıdır.

Authorization field'ı içeren request'lere verilen response'lar önbelleğe alınamaz.

10.3 Content-Encoding

Content-Encoding entity-header field'ı, media type için bir değiştirici olarak kullanılır. Mevcut olduğunda, kaynağa hangi ek content coding'in uygulandığını ve dolayısıyla Content-Type header field'ı tarafından başvurulan media type'ı elde etmek için hangi çözümleme mekanizmasının uygulanması gerektiğini belirtir. Content-Encoding öncelikle bir belgenin, temel media type kimliğini kaybetmeden sıkıştırılmasına olanak tanımak için kullanılır.

Content-Encoding = "Content-Encoding" ":" content-coding

Content coding'ler Bölüm 3.5'te tanımlanmıştır. Kullanım örneği: Content-Encoding: x-gzip

Content-Encoding, Request-URI tarafından tanımlanan kaynağın bir özelliğidir. Kaynak tipik olarak bu kodlamayla depolanır ve yalnızca işleme tabi tutulmadan önce çözümlenir.

10.4 Content-Length

Content-Length entity-header field'ı, alıcıya gönderilen Entity-Body'nin ondalık oktet sayısı cinsinden boyutunu ya da HEAD method'u söz konusu olduğunda request GET olsaydı gönderilecek Entity-Body'nin boyutunu belirtir.

*Content-Length = "Content-Length" ":" 1*DIGIT*

Örnek: Content-Length: 3495

Uygulamalar, entity'nin media type'ından bağımsız olarak aktarılacak Entity-Body'nin boyutunu belirtmek için bu field'ı kullanmalıdır. Entity body içeren tüm HTTP/1.0 request message'larında geçerli bir Content-Length field değeri zorunludur.

Sıfır veya sıfırdan büyük herhangi bir Content-Length değeri geçerlidir. Content-Length verilmediğinde bir response entity body'sinin uzunluğunun nasıl belirleneceği Bölüm 7.2.2'de açıklanmıştır.

Not: Bu field'ın anlamı, isteğe bağlı bir field olarak "message/external-body" content type içinde kullanıldığı MIME'deki karşılık gelen tanımdan önemli ölçüde farklıdır. HTTP'de entity'nin uzunluğu aktarılmadan önce belirlenebiliyorsa bu field her zaman kullanılmalıdır.

10.5 Content-Type

Content-Type entity-header field'ı, alıcıya gönderilen Entity-Body'nin media type'ını ya da HEAD method'u söz konusu olduğunda request GET olsaydı gönderilecek media type'ı belirtir.

Content-Type = "Content-Type" ":" media-type

Media type'lar Bölüm 3.6'da tanımlanmıştır. Field'a örnek: Content-Type: text/html

Bir entity'nin media type'ının belirlenmesine yönelik yöntemlere ilişkin ayrıntılı tartışma Bölüm 7.2.1'de yer almaktadır.

10.6 Date

Date general-header field'ı, message'ın oluşturulduğu tarih ve saati temsil eder; RFC 822'deki orig-date ile aynı anlambilime sahiptir. Field değeri, Bölüm 3.3'te açıklandığı üzere bir HTTP-date'tir.

Date = "Date" ":" HTTP-date

Örnek: Date: Tue, 15 Nov 1994 08:12:31 GMT

Bir message, user agent ile doğrudan connection (request durumunda) veya origin server ile doğrudan connection (response durumunda) üzerinden alınıyorsa tarih, alıcı taraftaki mevcut tarih olarak kabul edilebilir. Ancak origin tarafından bilinen tarih, önbelleğe alınmış response'ların değerlendirilmesi açısından önemli olduğundan origin server'lar her zaman bir Date header'ı eklemelidir. Client'lar yalnızca entity body içeren message'lara Date header field'ı eklemeli (POST request'inde olduğu gibi) ve bu bile isteğe bağlıdır. Date header field'ı bulunmayan alınan bir message, alıcı tarafından önbelleğe alınacaksa veya Date gerektiren bir protokol üzerinden gateway'e iletilecekse alıcı tarafından bir tarih atanmalıdır.

Teorik olarak tarih, entity'nin oluşturulduğu anı temsil etmelidir. Pratikte tarih, anlambilimsel değerini etkilemeksizin message oluşturma sırasında herhangi bir anda üretilebilir.

Not: Bu belgenin önceki bir versiyonu hatalı biçimde bu field'ın ekteki Entity-Body'nin oluşturulma tarihini içermesi gerektiğini belirtmiştir. Bu durum, gerçek (ve doğru) kullanımı yansıtacak şekilde değiştirilmiştir.

10.7 Expires

Expires entity-header field'ı, entity'nin bayat sayılması gereken tarih/saati belirtir. Bu, bilgi sağlayıcıların kaynağın değişkenliğini veya bilginin artık geçerli olmayabileceği bir tarihi önermesine olanak tanır. Uygulamalar bu entity'yi verilen tarihin ötesinde önbelleğe almamalıdır. Expires field'ının varlığı, özgün kaynağın o tarihte, öncesinde veya sonrasında değişeceği ya da var olmayı durduracağı anlamına gelmez. Ancak bir kaynağın belirli bir

tarihe kadar değişeceğini bilen veya tahmin eden bilgi sağlayıcılar, o tarihi içeren bir Expires header'ı eklemelidir. Format, Bölüm 3.3'teki HTTP-date tarafından tanımlanan mutlak tarih ve saattir.

Expires = "Expires" ":" HTTP-date

Kullanım örneği: Expires: Thu, 01 Dec 1994 16:00:00 GMT

Verilen tarih, Date header'ının değerine eşit veya daha önceyse alıcı ekteki entity'yi önbelleğe almamalıdır. Bir kaynak doğası gereği dinamikse (birçok veri üreten süreçte olduğu gibi) o kaynaktan gelen entity'lere, bu dinamizmi yansıtan uygun bir Expires değeri verilmelidir.

Expires field'ı, user agent'ı görünümünü yenilemeye veya bir kaynağı yeniden yüklemeye zorlamak için kullanılamaz; anlambilimi yalnızca önbellekleme mekanizmaları için geçerlidir ve bu tür mekanizmaların yalnızca söz konusu kaynak için yeni bir request başlatıldığında kaynağın sona erme durumunu kontrol etmesi gerekir.

User agent'lar çoğunlukla "Geri" düğmeleri ve geçmiş listeleri gibi, önceki oturumda alınan entity'leri yeniden görüntülemekte kullanılabilen geçmiş mekanizmalarına sahiptir. Varsayılan olarak Expires field'ı geçmiş mekanizmaları için geçerli değildir. Entity hâlâ depolanıyorsa geçmiş mekanizması, kullanıcı sona ermiş geçmiş belgelerini yenileme amacıyla agent'ı özellikle yapılandırmadığı sürece entity sona ermiş olsa bile onu görüntülemelidir.

Not: Uygulamaların Expires header'ının hatalı veya yanlış bilgili gerçeklemelerine karşı toleranslı olmaları önerilmektedir. Sıfır (0) değeri veya geçersiz bir tarih formatı, "hemen sona eriyor" ile eşdeğer kabul edilmelidir. Bu değerler HTTP/1.0 için yasal olmasa da sağlam bir gerçekleme her zaman tercih edilir.

10.8 From

From request-header field'ı, verilmişse request yapan user agent'ı denetleyen gerçek kullanıcının İnternet e-posta adresini içermelidir. Adres RFC 822 [7]'deki mailbox tarafından tanımlandığı şekilde (RFC 1123 [6] ile güncellenmiştir) makine tarafından kullanılabilir olmalıdır:

From = "From" ":" mailbox

Örnek: From: webmaster@w3.org

Bu header field'ı günlük tutma amacıyla ve geçersiz veya istenmeyen request'lerin kaynağını tanımlamak için kullanılabilir. Güvensiz bir erişim koruması biçimi olarak kullanılmamalıdır. Bu field'ın yorumu, request'in belirtilen kişi adına gerçekleştirildiği ve bu kişinin gerçekleştirilen method'un sorumluluğunu üstlendiği şeklindedir. Özellikle robot agent'lar bu header'ı eklemelidir; böylece alıcı tarafta sorunlar yaşandığında robotu çalıştırmaktan sorumlu kişiye ulaşılabilir.

Bu field'daki İnternet e-posta adresi, request'i gönderen İnternet host'undan farklı olabilir. Örneğin bir request proxy üzerinden geçirildiğinde özgün gönderenin adresi kullanılmalıdır.

Not: Client, From header field'ını kullanıcının onayı olmadan gönderme. Zira bu, kullanıcının gizlilik çıkarlarıyla veya sitenin güvenlik politikasıyla çelişebilir. Kullanıcının bir request öncesinde bu field'ın değerini istediği zaman devre dışı bırakabilmesi, etkinleştirebilmesi ve değiştirebilmesi önemle tavsiye edilir.

10.9 If-Modified-Since

If-Modified-Since request-header field'ı, GET method'unu koşullu hale getirmek için kullanılır: request edilen kaynak bu field'da belirtilen saatten bu yana değiştirilmemişse server'dan kaynağın bir kopyası döndürülmez; bunun yerine herhangi bir Entity-Body içermeyen 304 (değiştirilmemiş) response'u gönderilir.

If-Modified-Since = "If-Modified-Since" ":" HTTP-date

Field'a örnek: If-Modified-Since: Sat, 29 Oct 1994 19:43:31 GMT

Koşullu GET method'u, tanımlanan kaynağın yalnızca If-Modified-Since header'ında verilen tarihten bu yana değiştirilmişse aktarılmasını talep eder. Bunu belirleyen algoritma aşağıdaki durumları kapsar:

a) Request normalde 200 (tamam) dışında bir status ile sonuçlanacaksa veya geçilen If-Modified-Since tarihi geçersizse response, normal bir GET için olduğuyla birebir aynıdır. Server'ın mevcut zamanından sonraki bir tarih geçersizdir.

b) Kaynak If-Modified-Since tarihinden bu yana değiştirilmişse response, normal bir GET için olduğuyla birebir aynıdır.

c) Kaynak geçerli bir If-Modified-Since tarihinden bu yana değiştirilmemişse server 304 (değiştirilmemiş) response'u döndürmelidir.

Bu özelliğin amacı, minimum transaction yüküyle önbelleğe alınmış bilgilerin verimli biçimde güncellenmesine olanak tanımaktır.

10.10 Last-Modified

Last-Modified entity-header field'ı, gönderenin kaynağın en son ne zaman değiştirildiğini düşündüğü tarih ve saati belirtir. Bu field'ın tam anlambilimi, alıcının bunu nasıl yorumlaması gerektiği açısından tanımlanır: alıcının bu kaynağın Last-Modified field'ında verilen tarihten daha eski bir kopyası varsa bu kopya bayat kabul edilmelidir.

Last-Modified = "Last-Modified" ":" HTTP-date

Kullanım örneği: Last-Modified: Tue, 15 Nov 1994 12:45:26 GMT

Bu header field'ının tam anlamı gönderenin gerçeklemesine ve özgün kaynağın niteliğine bağlıdır. Dosyalar için bu yalnızca dosya sisteminin son değiştirilme zamanı olabilir. Dinamik olarak dahil edilen parçalar içeren entity'ler için, bileşen parçalarının son değiştirilme zamanlarından en güncel olanı olabilir. Veritabanı gateway'leri için kaydın son güncelleme zaman damgası olabilir. Sanal nesneler için dahili durumun en son değiştiği zaman olabilir.

Bir origin server, server'ın mesaj oluşturma zamanından daha sonraki bir Last-Modified tarihi göndermemelidir. Kaynağın son değiştirilmesinin gelecekteki bir zamana işaret edeceği bu tür durumlarda server, söz konusu tarihi mesaj oluşturma tarihiyle değiştirmelidir.

10.11 Location

Location response-header field'ı, Request-URI tarafından tanımlanan kaynağın tam konumunu tanımlar. 3xx response'ları için konum, kaynağa otomatik yönlendirme için server'ın tercih ettiği URL'yi belirtmelidir. Yalnızca tek bir mutlak URL'ye izin verilir.

Location = "Location" ":" absoluteURI

Örnek: Location: http://www.w3.org/hypertext/WWW/NewLocation.html

10.12 Pragma

Pragma general-header field'ı, request/response zinciri boyunca herhangi bir alıcı için geçerli olabilecek gerçeklemeye özgü direktifler eklemek için kullanılır. Tüm pragma direktifleri, protokol açısından isteğe bağlı davranışı belirtir; ancak bazı sistemler davranışın direktiflerle tutarlı olmasını gerektirebilir.

Pragma = "Pragma" ":" 1#pragma-directive pragma-directive = "no-cache" | extension-pragma extension-pragma = token ["=" word]

Request message'ında "no-cache" direktifi mevcutsa uygulama, request edilen şeyin önbelleğe alınmış bir kopyasına sahip olsa bile request'i origin server'a iletmelidir. Bu, client'ın request'ine yetkili bir response almasında ısrar etmesine olanak tanır. Aynı zamanda client'ın bozuk veya bayat olduğu bilinen önbelleğe alınmış bir kopyayı yenilemesine de imkân verir.

Pragma direktifleri, söz konusu uygulamalar için önemi ne olursa olsun, proxy veya gateway uygulaması tarafından aktarılmalıdır; zira direktifler, request/response zinciri boyunca tüm alıcılar için geçerli olabilir. Belirli bir alıcı için pragma belirtmek mümkün değildir; ancak alıcıyla ilgisi olmayan pragma direktifleri o alıcı tarafından göz ardı edilmelidir.

10.13 Referer

Referer request-header field'ı, server'ın yararına, client'ın Request-URI'nin alındığı kaynağın adresini (URI) belirtmesine olanak tanır. Bu, server'ın ilgi amacıyla, günlük tutma, optimize edilmiş önbellekleme vb. için kaynaklara geri bağlantı listeleri oluşturmasına imkân tanır. Ayrıca bakım amacıyla geçerliliğini yitirmiş veya yanlış yazılmış bağlantıların izlenmesine olanak verir. Request-URI, klavye girişi gibi kendi URI'si olmayan bir kaynaktan elde edilmişse Referer field'ı gönderilmemelidir.

Referer = "Referer" ":" (absoluteURI | relativeURI)

Örnek: Referer: http://www.w3.org/hypertext/DataSources/Overview.html

Kısmi bir URI verilmişse Request-URI'ye göre yorumlanmalıdır. URI bir fragment içermemelidir.

Not: Bir bağlantının kaynağı özel bilgi olabileceğinden veya başka türlü özel bir bilgi kaynağını açığa çıkarabileceğinden, kullanıcının Referer field'ının gönderilip gönderilmeyeceğini seçebilmesi kuvvetle tavsiye edilir. Örneğin bir tarayıcı client'ı, sırasıyla Referer ve From bilgisinin gönderilmesini etkinleştiren/devre dışı bırakan, açık/anonim gezinti için bir geçiş düğmesine sahip olabilir.

10.14 Server

Server response-header field'ı, request'i işlemek için origin server tarafından kullanılan yazılım hakkında bilgi içerir. Field, server'ı ve önemli alt ürünleri tanımlayan birden fazla product token'ı (Bölüm 3.7) ve yorum içerebilir. Gelenek olarak product token'ları, uygulamayı tanımlamadaki önem derecelerine göre sıralanır.

Server = "Server" ":" 1(product | comment)*

Örnek: Server: CERN/3.0 libwww/2.17

Response bir proxy üzerinden iletiliyorsa proxy uygulaması kendi verilerini ürün listesine eklememelidir.

Not: Server'ın belirli yazılım versiyonunu açıklamak, server makinesinin güvenlik açıkları içerdiği bilinen yazılımlara yönelik saldırılara karşı daha savunmasız hale gelmesine yol açabilir. Server gerçekleyicileri bu field'ı yapılandırılabilir bir seçenek haline getirmeye teşvik edilmektedir.

Not: Bazı mevcut server'lar Server field'ı içinde kendilerini product token sözdizimiyle sınırlandıramamaktadır.

10.15 User-Agent

User-Agent request-header field'ı, request'i başlatan user agent hakkında bilgi içerir. Bu bilgi istatistiksel amaçlarla, protokol ihlallerinin izlenmesi ve belirli user agent kısıtlamalarından kaçınmak için response'ları uyarlamak amacıyla user agent'ların otomatik tanınmasına yöneliktir. Zorunlu olmasa da user agent'lar bu field'ı request'lere eklemelidir. Field, agent'ı ve user agent'ın önemli bir bölümünü oluşturan alt ürünleri tanımlayan birden fazla product token'ı (Bölüm 3.7) ve yorum içerebilir. Gelenek olarak product token'ları, uygulamayı tanımlamadaki önem derecelerine göre sıralanır.

User-Agent = "User-Agent" ":" 1(product | comment)*

Örnek: User-Agent: CERN-LineMode/2.15 libwww/2.17b3

Not: Bazı mevcut proxy uygulamaları kendi ürün bilgilerini User-Agent field'ındaki listeye eklemektedir. Bu önerilmez; zira bu field'ların makine tarafından yorumlanmasını belirsiz hale getirir.

Not: Bazı mevcut client'lar User-Agent field'ı içinde kendilerini product token sözdizimiyle sınırlandıramamaktadır.

10.16 WWW-Authenticate

WWW-Authenticate response-header field'ı, 401 (yetkisiz) response message'larına eklenmelidir. Field değeri, Request-URI için geçerli kimlik doğrulama scheme'ini ve parametrelerini belirten en az bir meydan okuma içermelidir.

WWW-Authenticate = "WWW-Authenticate" ":" 1#challenge

HTTP erişim kimlik doğrulama süreci Bölüm 11'de açıklanmıştır. Bir meydan okumanın içeriği kendi başına virgülle ayrılmış bir kimlik doğrulama parametreleri listesi içerebileceğinden, WWW-Authenticate field değeri birden fazla meydan okuma içeriyorsa veya birden fazla WWW-Authenticate header field'ı sağlanmışsa user agent'lar bunu ayrıştırırken özellikle dikkatli olmalıdır.

11. Erişim Kimlik Doğrulaması

HTTP, bir server'ın client request'ine meydan okumak ve client'ın kimlik doğrulama bilgisi sağlamak için kullanabileceği basit bir meydan okuma-yanıt kimlik doğrulama mekanizması sunar. Kimlik doğrulama scheme'ini tanımlamak için genişletilebilir, büyük/küçük harf duyarsız bir token kullanır; ardından söz konusu scheme aracılığıyla kimlik doğrulamayı sağlamak için gereken parametreleri taşıyan virgülle ayrılmış özellik-değer çiftlerinin listesi gelir.

auth-scheme = token auth-param = token "=" quoted-string

401 (yetkisiz) response message'ı, bir origin server tarafından user agent'ın yetkilendirmesine meydan okumak için kullanılır. Bu response, request edilen kaynak için geçerli en az bir meydan okuma içeren bir WWW-Authenticate header field'ı içermelidir.

*challenge = auth-scheme 1*SP realm *("," auth-param) realm = "realm" "=" realm-value realm-value = quoted-string*

realm özelliği (büyük/küçük harf duyarsız), meydan okuma yayımlayan tüm kimlik doğrulama scheme'leri için zorunludur. realm değeri (büyük/küçük harfe duyarlı), erişilen server'ın kanonik kök URL'siyle birlikte koruma alanını tanımlar. Bu realm'ler, bir server üzerindeki korumalı kaynakların her birinin kendi kimlik doğrulama scheme'i ve/veya yetkilendirme veritabanına sahip olduğu bir koruma alanı kümesine bölünmesine olanak tanır. realm değeri, genellikle origin server tarafından atanan ve kimlik doğrulama scheme'ine özgü ek anlambilime sahip olabilen bir dizedir.

Bir server ile kimliğini doğrulamak isteyen user agent (genellikle, ancak zorunlu olarak değil, 401 response aldıktan sonra) bunu request'e Authorization header field'ı ekleyerek yapabilir. Authorization field değeri, request edilen kaynağın realm'i için user agent'ın kimlik doğrulama bilgilerini içeren kimlik bilgilerinden oluşur.

credentials = basic-credentials | (auth-scheme #auth-param)

Kimlik bilgilerinin bir user agent tarafından otomatik olarak uygulanabileceği alan, koruma alanı tarafından belirlenir. Önceki bir request yetkilendirilmişse aynı kimlik bilgileri, kimlik doğrulama scheme'i, parametreler ve/veya kullanıcı tercihi tarafından belirlenen süre boyunca o koruma alanındaki diğer tüm request'ler için yeniden kullanılabilir. Kimlik

doğrulama scheme'i tarafından aksi belirtilmedikçe tek bir koruma alanı kendi server'ının kapsamını aşamaz.

Server, bir request ile gönderilen kimlik bilgilerini kabul etmek istemiyorsa 403 (yasak) response'u döndürmelidir.

HTTP protokolü, uygulamaları erişim kimlik doğrulaması için yalnızca bu basit meydan okuma-yanıt mekanizmasıyla sınırlandırmaz. Transport katmanında şifreleme veya mesaj kapsülleme gibi ek mekanizmalar ile kimlik doğrulama bilgilerini belirten ek header field'ları kullanılabilir. Ancak bu ek mekanizmalar bu spesifikasyon tarafından tanımlanmamaktadır.

Proxy'ler user agent kimlik doğrulaması konusunda tamamen şeffaf olmalıdır. Yani WWW-Authenticate ve Authorization header'larını değiştirmeksizin iletmeli ve Authorization içeren bir request'e verilen response'u önbelleğe almamalıdır. HTTP/1.0, client'ın bir proxy ile kimlik doğrulaması yapması için herhangi bir yol sağlamaz.

11.1 Temel Kimlik Doğrulama Scheme'i

"basic" kimlik doğrulama scheme'i, user agent'ın her realm için bir kullanıcı kimliği ve parola ile kimliğini doğrulaması gerektiği modeline dayanır. realm değeri, yalnızca aynı server üzerindeki diğer realm'lerle eşitlik açısından karşılaştırılabilen opak bir dize olarak kabul edilmelidir. Server, Request-URI'nin koruma alanı için kullanıcı kimliğini ve parolayı doğrulayabilirse request'i yetkilendirir. İsteğe bağlı kimlik doğrulama parametresi bulunmamaktadır.

Koruma alanındaki bir URI için yetkisiz bir request alındığında server aşağıdaki gibi bir meydan okumayla yanıt vermelidir:

WWW-Authenticate: Basic realm="WallyWorld"

Burada "WallyWorld", server tarafından Request-URI'nin koruma alanını tanımlamak için atanan dizedir.

Yetkilendirme almak için client, kullanıcı kimliği ve parolayı tek bir iki nokta üst üste (":") karakteriyle ayırarak kimlik bilgilerindeki base64 [5] kodlanmış bir dize içinde gönderir.

*basic-credentials = "Basic" SP basic-cookie basic-cookie = <base64 [5] kodlaması ile userid-password, 76 karakter/satır sınırı hariç> userid-password = [token] ":" *TEXT*

User agent "Aladdin" kullanıcı kimliğini ve "open sesame" parolasını göndermek istiyorsa aşağıdaki header field'ını kullanır:

Authorization: Basic QWxhZGRpbjpvcGVuIHNlc2FtZQ==

Temel kimlik doğrulama scheme'i, bir HTTP server'ındaki kaynaklara yetkisiz erişimi filtrelemenin güvensiz bir yöntemidir. Client ve server arasındaki connection'ın güvenilir bir taşıyıcı olarak kabul edilebileceği varsayımına dayanır. Bu durum açık bir ağda genellikle doğru olmadığından temel kimlik doğrulama scheme'i buna göre kullanılmalıdır. Buna karşın client'lar, bunu kullanan server'larla iletişim kurabilmek için scheme'i gerçeklemelidir.

12. Güvenlik Değerlendirmeleri

Bu bölüm, uygulama geliştiricilerini, bilgi sağlayıcıları ve kullanıcıları bu belge tarafından açıklanan HTTP/1.0'daki güvenlik sınırlamaları hakkında bilgilendirmeyi amaçlamaktadır. Tartışma, ortaya çıkan sorunlara kesin çözümler sunmamakla birlikte güvenlik risklerini azaltmaya yönelik bazı önerilerde bulunmaktadır.

12.1 Client Kimlik Doğrulaması

Bölüm 11.1'de belirtildiği üzere Temel kimlik doğrulama scheme'i, güvenli bir kullanıcı kimlik doğrulama yöntemi değildir; ayrıca Entity-Body'nin taşıyıcı olarak kullanılan fiziksel ağ üzerinden açık metin olarak iletilmesini engellemez. HTTP/1.0, güvenliği artırmak için ek kimlik doğrulama scheme'lerinin ve şifreleme mekanizmalarının kullanılmasını engellemez.

12.2 Güvenli Method'lar

Client yazılımının yazarları, yazılımın İnternet üzerindeki etkileşimlerde kullanıcıyı temsil ettiğinin ve kendileri veya başkaları için beklenmedik bir anlam taşıyabilecek eylemler konusunda kullanıcıyı bilinçlendirmeye özen göstermeleri gerektiğinin farkında olmalıdır.

Özellikle GET ve HEAD method'larının hiçbir zaman erişim dışında bir eylemde bulunma anlamı taşımaması gerektiği konusunda bir kural oluşturulmuştur. Bu method'lar "güvenli" kabul edilmelidir. Bu, user agent'ların POST gibi diğer method'ları özel biçimde temsil etmesine olanak tanır; böylece kullanıcı, olası güvensiz bir eylem talep edildiğinin farkında olur.

Doğal olarak server'ın GET request'ini gerçekleştirirken yan etkilere yol açmamasını sağlamak mümkün değildir; hatta bazı dinamik kaynaklar bunu bir özellik olarak kabul eder. Buradaki temel ayrım, kullanıcının yan etkileri talep etmemiş olmasıdır; dolayısıyla bunların sorumluluğu kullanıcıya yüklenemez.

12.3 Server Günlük Bilgisinin Kötüye Kullanımı

Bir server, kullanıcının okuma alışkanlıklarını veya ilgi alanlarını ortaya çıkarabilecek kişisel verileri request'ler aracılığıyla kaydedebilecek konumdadır. Bu bilgi açıkça gizli niteliktedir ve bazı ülkelerde işlenmesi yasal düzenlemelere tabi olabilir. HTTP protokolünü veri sağlamak amacıyla kullanan kişiler, yayımlanan sonuçlarla tanımlanabilir durumdaki bireylerin izni olmaksızın bu tür materyalin dağıtılmamasını sağlamakla yükümlüdür.

12.4 Hassas Bilgilerin Aktarımı

Herhangi bir genel veri aktarım protokolü gibi HTTP de aktarılan verinin içeriğini denetleyemez; ayrıca herhangi bir request bağlamında belirli bir bilginin hassasiyetini önceden belirlemenin herhangi bir yöntemi yoktur. Bu nedenle uygulamalar bu bilgi üzerinde mümkün olduğunca fazla denetim yetkisini bilgiyi sağlayan tarafa bırakmalıdır. Bu bağlamda özellikle değerlendirilmesi gereken üç header field'ı mevcuttur: Server, Referer ve From.

Server'ın belirli yazılım versiyonunu açıklamak, server makinesinin güvenlik açıkları içerdiği bilinen yazılımlara yönelik saldırılara karşı daha savunmasız hale gelmesine yol açabilir. Gerçekleyiciler Server header field'ını yapılandırılabilir bir seçenek haline getirmelidir.

Referer field'ı, okuma alışkanlıklarının incelenmesine ve ters bağlantıların çizilmesine olanak tanır. Son derece yararlı olabilmesine karşın, kullanıcı ayrıntıları Referer'daki bilgiden ayrılmadığı takdirde bu gücün kötüye kullanılması mümkündür. Kişisel bilgiler kaldırıldıktan sonra bile Referer field'ı, yayımlanması uygun olmayan özel bir belgenin URI'sini açığa çıkarabilir.

From field'ında gönderilen bilgi, kullanıcının gizlilik çıkarlarıyla veya sitenin güvenlik politikasıyla çelişebileceğinden kullanıcı bu field'ın içeriğini devre dışı bırakabilecek, etkinleştirebilecek ve değiştirebilecek durumda olmadan iletilmemelidir. Kullanıcı bu field'ın içeriğini kullanıcı tercihleri veya uygulama varsayılan yapılandırması aracılığıyla belirleyebilmelidir.

From ve Referer bilgilerinin gönderilmesini etkinleştirmek veya devre dışı bırakmak için kullanıcıya uygun bir geçiş arayüzü sağlanması önerilmekte; ancak zorunlu tutulmamaktadır.

12.5 Dosya ve Yol Adlarına Dayalı Saldırılar

HTTP origin server gerçekleyicileri, HTTP request'leri tarafından döndürülen belgeleri yalnızca server yöneticilerinin amaçladıklarıyla sınırlandırmaya özen göstermelidir. Bir HTTP server'ı HTTP URI'lerini doğrudan dosya sistemi çağrılarına çeviriyorsa HTTP client'larına iletilmesi amaçlanmayan dosyaları servis etmemeye özellikle dikkat etmelidir. Örneğin Unix, Microsoft Windows ve diğer işletim sistemleri, bir üst dizin düzeyini belirtmek için ".." yol bileşenini kullanır. Bu tür sistemlerde bir HTTP server'ı, Request-URI'de HTTP server'ı aracılığıyla erişilebilir olması amaçlananların dışındaki bir kaynağa erişime izin verecek herhangi bir bu tür yapıyı engellemelidir. Benzer şekilde, yalnızca server içinde kullanılmak üzere tasarlanmış dosyalar (erişim kontrol dosyaları, yapılandırma dosyaları ve script kodu gibi) hassas bilgiler içerebileceğinden uygunsuz erişime karşı korunmalıdır. Deneyimler, bu tür HTTP server gerçeklemelerindeki küçük hataların güvenlik risklerine dönüştüğünü ortaya koymuştur.

13. Teşekkürler

Bu spesifikasyon, David H. Crocker'ın RFC 822 [7] için tanımladığı genişletilmiş BNF ve genel yapıları yoğun biçimde kullanmaktadır. Benzer şekilde Nathaniel Borenstein ve Ned Freed'in MIME [5] için sağladığı tanımların büyük bölümünü yeniden kullanmaktadır. Bu tanımların spesifikasyona dahil edilmesinin, HTTP/1.0 ile İnternet posta mesaj formatları arasındaki ilişki konusundaki geçmişteki karışıklığı gidermeye yardımcı olacağını umuyoruz.

HTTP protokolü geçen dört yıl içinde önemli ölçüde evrim geçirmiştir. Geniş ve aktif bir geliştirici topluluğundan, www-talk posta listesine katılan pek çok kişiden yararlanmış; HTTP'nin ve genel olarak World-Wide Web'in başarısından en çok sorumlu olan da bu topluluk olmuştur. Marc Andreessen, Robert Cailliau, Daniel W. Connolly, Bob Denny, Jean-Francois Groff, Phillip M. Hallam-Baker, Hakon W. Lie, Ari Luotonen, Rob McCool, Lou Montulli, Dave Raggett, Tony Sanders ve Marc VanHeyningen, protokolün erken versiyonlarına ilişkin çeşitli yönlerin tanımlanmasındaki katkıları nedeniyle özellikle takdire değerdir.

Paul Hoffman, bu belgenin bilgilendirici statüsüne ilişkin bölümleri ve Ek C ile D'yi katkıda bulunmuştur.

Bu belge, HTTP-WG'ye katılan tüm kişilerin yorumlarından büyük ölçüde yararlanmıştır. Daha önce belirtilenlere ek olarak aşağıdaki kişiler bu spesifikasyona katkıda bulunmuştur:

Gary Adams, Harald Tveit Alvestrand, Keith Ball, Brian Behlendorf, Paul Burchard, Maurizio Codogno, Mike Cowlishaw, Roman Czyborra, Michael A. Dolan, John Franks, Jim Gettys, Marc Hedlund, Koen Holtman, Alex Hopmann, Bob Jernigan, Shel Kaphan, Martijn Koster, Dave Kristol, Daniel LaLiberte, Paul Leach, Albert Lunde, John C. Mallery, Larry Masinter, Mitra, Jeffrey Mogul, Gavin Nicol, Bill Perry, Jeffrey Perry, Owen Rees, Luigi Rizzo, David Robinson, Marc Salomon, Rich Salz, Jim Seidman, Chuck Shotton, Eric W. Sink, Simon E. Spero, Robert S. Thau, Francois Yergeau, Mary Ellen Zurko, Jean-Philippe Martin-Flatin.

14. Referanslar

[1] Anklesaria, F., McCahill, M., Lindner, P., Johnson, D., Torrey, D., ve B. Alberti, "The Internet Gopher Protocol: A Distributed Document Search and Retrieval Protocol", RFC 1436, University of Minnesota, Mart 1993.

[2] Berners-Lee, T., "Universal Resource Identifiers in WWW: A Unifying Syntax for the Expression of Names and Addresses of Objects on the Network as used in the World-Wide Web", RFC 1630, CERN, Haziran 1994.

[3] Berners-Lee, T., ve D. Connolly, "Hypertext Markup Language - 2.0", RFC 1866, MIT/W3C, Kasım 1995.

[4] Berners-Lee, T., Masinter, L., ve M. McCahill, "Uniform Resource Locators (URL)", RFC 1738, CERN, Xerox PARC, University of Minnesota, Aralık 1994.

[5] Borenstein, N., ve N. Freed, "MIME (Multipurpose Internet Mail Extensions) Part One: Mechanisms for Specifying and Describing the Format of Internet Message Bodies", RFC 1521, Bellcore, Innosoft, Eylül 1993.

[6] Braden, R., "Requirements for Internet hosts - Application and Support", STD 3, RFC 1123, IETF, Ekim 1989.

[7] Crocker, D., "Standard for the Format of ARPA Internet Text Messages", STD 11, RFC 822, UDEL, Ağustos 1982.

[8] F. Davis, B. Kahle, H. Morris, J. Salem, T. Shen, R. Wang, J. Sui, ve M. Grinbaum. "WAIS Interface Protocol Prototype Functional Specification." (v1.5), Thinking Machines Corporation, Nisan 1990.

[9] Fielding, R., "Relative Uniform Resource Locators", RFC 1808, UC Irvine, Haziran 1995.

[10] Horton, M., ve R. Adams, "Standard for interchange of USENET Messages", RFC 1036 (RFC 850'yi geçersiz kılar), AT&T Bell Laboratories, Center for Seismic Studies, Aralık 1987.

[11] Kantor, B., ve P. Lapsley, "Network News Transfer Protocol: A Proposed Standard for the Stream-Based Transmission of News", RFC 977, UC San Diego, UC Berkeley, Şubat 1986.

[12] Postel, J., "Simple Mail Transfer Protocol." STD 10, RFC 821, USC/ISI, Ağustos 1982.

[13] Postel, J., "Media Type Registration Procedure." RFC 1590, USC/ISI, Mart 1994.

[14] Postel, J., ve J. Reynolds, "File Transfer Protocol (FTP)", STD 9, RFC 959, USC/ISI, Ekim 1985.

[15] Reynolds, J., ve J. Postel, "Assigned Numbers", STD 2, RFC 1700, USC/ISI, Ekim 1994.

[16] Sollins, K., ve L. Masinter, "Functional Requirements for Uniform Resource Names", RFC 1737, MIT/LCS, Xerox Corporation, Aralık 1994.

[17] US-ASCII. Coded Character Set - 7-Bit American Standard Code for Information Interchange. Standard ANSI X3.4-1986, ANSI, 1986.

[18] ISO-8859. International Standard -- Information Processing -- 8-bit Single-Byte Coded Graphic Character Sets -- Bölüm 1: Latin alfabesi No. 1, ISO 8859-1:1987. Bölüm 2: Latin alfabesi No. 2, ISO 8859-2, 1987. Bölüm 3: Latin alfabesi No. 3, ISO 8859-3, 1988. Bölüm 4: Latin alfabesi No. 4, ISO 8859-4, 1988. Bölüm 5: Latin/Kiril alfabesi, ISO 8859-5, 1988. Bölüm 6: Latin/Arap alfabesi, ISO 8859-6, 1987. Bölüm 7: Latin/Yunan alfabesi, ISO 8859-7, 1987. Bölüm 8: Latin/İbrani alfabesi, ISO 8859-8, 1988. Bölüm 9: Latin alfabesi No. 5, ISO 8859-9, 1990.

15. Yazar Adresleri

Tim Berners-Lee Direktör, W3 Consortium MIT Laboratory for Computer Science 545 Technology Square Cambridge, MA 02139, U.S.A. Faks: +1 (617) 258 8682 E-posta: timbl@w3.org

Roy T. Fielding Department of Information and Computer Science University of California Irvine, CA 92717-3425, U.S.A. Faks: +1 (714) 824-4056 E-posta: fielding@ics.uci.edu

Henrik Frystyk Nielsen W3 Consortium MIT Laboratory for Computer Science 545 Technology Square Cambridge, MA 02139, U.S.A. Faks: +1 (617) 258 8682 E-posta: frystyk@w3.org

Ekler

Bu ekler yalnızca bilgilendirme amacıyla sunulmuştur; HTTP/1.0 spesifikasyonunun bir parçasını oluşturmamaktadır.

A. İnternet Media Type message/http

Bu belge, HTTP/1.0 protokolünü tanımlamanın yanı sıra "message/http" İnternet media type'ının spesifikasyonu olarak da hizmet vermektedir. Aşağıdakiler IANA [13]'ya kaydedilecektir.

Media Type adı: message Media subtype adı: http Zorunlu parametreler: yok İsteğe bağlı parametreler: version, msgtype version: İçerilen message'ın HTTP-Version numarası (örn.

"1.0"). Mevcut değilse versiyon body'nin ilk satırından belirlenebilir. msgtype: Message type'ı — "request" veya "response". Mevcut değilse type, body'nin ilk satırından belirlenebilir. Kodlama değerlendirmeleri: yalnızca "7bit", "8bit" veya "binary"ye izin verilir. Güvenlik değerlendirmeleri: yok

B. Toleranslı Uygulamalar

Bu belge, HTTP/1.0 message'larının oluşturulmasına ilişkin gereksinimleri belirtse de tüm uygulamalar gerçeklemelerinde doğru olmayacaktır. Bu nedenle işlevsel uygulamaların, sapmaların belirsiz biçimde yorumlanabileceği durumlarda bu sapmalara toleranslı olması önerilmektedir.

Client'lar Status-Line'ı ayrıştırırken, server'lar ise Request-Line'ı ayrıştırırken toleranslı olmalıdır. Özellikle yalnızca tek bir SP gerekli olsa da field'lar arasında herhangi bir miktarda SP veya HT karakterini kabul etmelidirler.

HTTP header field'ları için satır sonlandırıcı CRLF dizisidir. Ancak uygulamaların bu tür header'ları ayrıştırırken tek bir LF'yi satır sonlandırıcı olarak tanıması ve baştaki CR'yi görmezden gelmesi önerilmektedir.

C. MIME ile İlişkisi

HTTP/1.0, entity'lerin açık çeşitlilikte gösterimlerle ve genişletilebilir mekanizmalarla iletilmesine olanak tanımak için İnternet Mail (RFC 822 [7]) ve Multipurpose Internet Mail Extensions (MIME [5]) için tanımlanmış yapıların büyük bölümünü kullanır. Ancak RFC 1521 posta konusunu ele almaktadır ve HTTP'nin RFC 1521'de tanımlananlardan farklı birkaç özelliği mevcuttur. Bu farklılıklar, binary connection'lar üzerindeki performansı optimize etmek, yeni media type'larının kullanımında daha fazla özgürlük tanımak, tarih karşılaştırmalarını kolaylaştırmak ve bazı erken HTTP server ile client'larının pratiklerini kabul etmek amacıyla özenle seçilmiştir.

Bu yazı hazırlanırken RFC 1521'in revize edileceği öngörülmektedir. Revizyonlar, HTTP/1.0'da bulunan ancak RFC 1521'de yer almayan bazı pratikleri içerebilir.

Bu ek, HTTP'nin RFC 1521'den farklılaştığı belirli alanları açıklamaktadır. Katı MIME ortamlarına açılan proxy ve gateway'ler bu farklılıkların farkında olmalı ve gerektiğinde uygun dönüşümleri sağlamalıdır. MIME ortamlarından HTTP'ye açılan proxy ve gateway'lerin de bazı dönüşümler gerekebileceğinden bu farklılıklardan haberdar olması gerekmektedir.

C.1 Kanonik Forma Dönüştürme

RFC 1521, RFC 1521 [5]'in G Eki'nde açıklandığı üzere bir İnternet posta entity'sinin aktarılmadan önce kanonik forma dönüştürülmesini zorunlu kılar. Bu belgenin 3.6.1. Bölümü, HTTP üzerinden iletildiğinde "text" media type'ının subtype'larına izin verilen biçimleri açıklamaktadır.

RFC 1521, "text" Content-Type'ına sahip içeriğin satır sonlarını CRLF olarak temsil etmesini ve satır sonu dizileri dışında CR veya LF kullanılmasını yasaklar. HTTP, bir message HTTP üzerinden iletildiğinde metin içeriğinde satır sonunu belirtmek için CRLF, yalnız CR ve yalnız LF kullanımına izin verir.

Mümkün olduğunda HTTP'den katı RFC 1521 ortamına açılan bir proxy veya gateway, bu belgenin 3.6.1. Bölümünde açıklanan metin media type'larındaki tüm satır sonlarını RFC 1521 kanonik CRLF biçimine çevirmelidir. Ancak bu işlemin, Content-Encoding varlığı ve HTTP'nin bazı çok baytlı karakter setlerinde olduğu gibi CR ve LF'yi temsil etmek için 13 ve 10. oktetleri kullanmayan karakter setlerinin kullanımına izin vermesi nedeniyle karmaşıklaşabileceğine dikkat edilmelidir.

C.2 Tarih Formatlarının Dönüştürülmesi

HTTP/1.0, tarih karşılaştırma sürecini basitleştirmek amacıyla sınırlı bir tarih formatları kümesi kullanır (Bölüm 3.3). Diğer protokollerden açılan proxy ve gateway'ler, bir message'daki Date header field'ının HTTP/1.0 formatlarından birine uyduğundan emin olmalı ve gerekirse tarihi yeniden yazmalıdır.

C.3 Content-Encoding'in Tanıtımı

RFC 1521, HTTP/1.0'ın Content-Encoding header field'ına eşdeğer herhangi bir kavramı içermez. Bu, media type için bir değiştirici olarak işlev gördüğünden HTTP'den MIME uyumlu protokollere açılan proxy ve gateway'ler, message'ı iletmeden önce ya Content-Type header field'ının değerini değiştirmeli ya da Entity-Body'yi çözümlemelidir. (İnternet posta için Content-Type'ın bazı deneysel uygulamaları, Content-Encoding ile eşdeğer bir işlev gerçekleştirmek amacıyla ";conversions=<content-coding>" biçiminde bir media-type parametresi kullanmıştır. Ancak bu parametre RFC 1521'nin bir parçası değildir.)

C.4 Content-Transfer-Encoding Yok

HTTP, RFC 1521'in Content-Transfer-Encoding (CTE) field'ını kullanmaz. MIME uyumlu protokollerden HTTP'ye açılan proxy ve gateway'ler, response message'ını HTTP client'a iletmeden önce kimlik dışı CTE ("quoted-printable" veya "base64") kodlamasını kaldırmalıdır.

HTTP'den MIME uyumlu protokollere açılan proxy ve gateway'ler, message'ın söz konusu protokol üzerinde güvenli aktarım için doğru format ve kodlamada olmasını sağlamaktan sorumludur; burada "güvenli aktarım" kullanılan protokolün sınırlamaları tarafından tanımlanmaktadır. Bu tür proxy veya gateway, hedef protokol üzerinden güvenli aktarım olasılığını artırıyorsa veriyi uygun bir Content-Transfer-Encoding ile etiketlemelidir.

C.5 Multipart Body Parçalarındaki HTTP Header Field'ları

RFC 1521'de, multipart body parçalarındaki header field'larının büyük bölümü, field adı "Content-" ile başlamadığı sürece genellikle göz ardı edilir. HTTP/1.0'da ise multipart body parçaları, o parçanın anlamı açısından önem taşıyan herhangi bir HTTP header field'ı içerebilir.

D. Ek Özellikler

Bu ek, bazı mevcut HTTP gerçeklemeleri tarafından kullanılan ancak çoğu HTTP/1.0 uygulamasında tutarlı ve doğru biçimde hayata geçirilmeyen protokol öğelerini belgelemektedir. Gerçekleyiciler bu özelliklerden haberdar olmalı; ancak bunların diğer HTTP/1.0 uygulamalarında mevcut olacağına veya bu uygulamalarla birlikte çalışabileceğine güvenmemelidir.

D.1 Ek Request Method'ları

D.1.1 PUT

PUT method'u, ekteki entity'nin belirtilen Request-URI altında depolanmasını talep eder. Request-URI halihazırda var olan bir kaynağa atıfta bulunuyorsa ekteki entity, origin server'da bulunanın değiştirilmiş bir versiyonu olarak kabul edilmelidir. Request-URI mevcut bir kaynağa işaret etmiyorsa ve söz konusu URI, request yapan user agent tarafından yeni bir kaynak olarak tanımlanabilecek durumdaysa origin server o URI ile kaynağı oluşturabilir.

POST ve PUT request'leri arasındaki temel fark, Request-URI'nin farklı anlamlarına yansımaktadır. POST request'indeki URI, ekteki entity'yi işlenecek veri olarak ele alacak kaynağı tanımlar. Bu kaynak veri kabul eden bir süreç, başka bir protokole açılan gateway veya açıklamaları kabul eden ayrı bir entity olabilir. Buna karşılık PUT request'indeki URI, request ile birlikte gönderilen entity'yi tanımlar; user agent hangi URI'nin amaçlandığını bilir ve server, request'i başka bir kaynağa uygulamamalıdır.

D.1.2 DELETE

DELETE method'u, origin server'dan Request-URI tarafından tanımlanan kaynağı silmesini talep eder.

D.1.3 LINK

LINK method'u, Request-URI tarafından tanımlanan mevcut kaynak ile diğer mevcut kaynaklar arasında bir veya daha fazla Link ilişkisi kurar.

D.1.4 UNLINK

UNLINK method'u, Request-URI tarafından tanımlanan mevcut kaynaktan bir veya daha fazla Link ilişkisini kaldırır.

D.2 Ek Header Field Tanımları

D.2.1 Accept

Accept request-header field'ı, request'e yanıt olarak kabul edilebilir media range'lerin listesini belirtmek için kullanılabilir. Yıldız "" karakteri, media type'larını gruplara ayırmak için kullanılır; "/" tüm media type'larını, "type/" ise o type'ın tüm subtype'larını ifade eder. Client tarafından verilen range'ler kümesi, request'in bağlamı göz önüne alındığında kabul edilebilir type'ları temsil etmelidir.

D.2.2 Accept-Charset

Accept-Charset request-header field'ı, varsayılan US-ASCII ve ISO-8859-1 dışında tercih edilen karakter setlerinin listesini belirtmek için kullanılabilir. Bu field, daha kapsamlı veya özel amaçlı karakter setlerini anlayabilen client'ların bu kapasiteyi, belgeleri söz konusu karakter setlerinde sunan server'lara bildirmesine olanak tanır.

D.2.3 Accept-Encoding

Accept-Encoding request-header field'ı, Accept field'ına benzer; ancak response'da kabul edilebilir content-coding değerlerini kısıtlar.

D.2.4 Accept-Language

Accept-Language request-header field'ı, Accept field'ına benzer; ancak request'e yanıt olarak tercih edilen doğal dillerin kümesini kısıtlar.

D.2.5 Content-Language

Content-Language entity-header field'ı, ekteki entity'nin amaçlanan kitlesinin doğal dilini veya dillerini açıklar. Bunun entity içinde kullanılan tüm dillerle eşdeğer olmayabileceğine dikkat edilmelidir.

D.2.6 Link

Link entity-header field'ı, entity ile başka bir kaynak arasındaki ilişkiyi tanımlamanın bir yolunu sağlar. Bir entity birden fazla Link değeri içerebilir. Metabilgi düzeyindeki bağlantılar tipik olarak hiyerarşik yapı ve gezinti yolları gibi ilişkileri belirtir.

D.2.7 MIME-Version

HTTP message'ları, message'ı oluşturmak için hangi MIME protokol versiyonunun kullanıldığını belirtmek amacıyla tek bir MIME-Version general-header field'ı içerebilir. RFC 1521 [5] tarafından tanımlandığı şekliyle MIME-Version header field'ının kullanımı, message'ın MIME uyumlu olduğunu göstermelidir. Ne var ki bazı eski HTTP/1.0 server'ları bunu rastgele biçimde göndermektedir; bu nedenle bu field göz ardı edilmelidir.

D.2.8 Retry-After

Retry-After response-header field'ı, servisin request yapan client için ne kadar süre kullanılamayacağını belirtmek amacıyla 503 (servis kullanılamıyor) response'uyla birlikte kullanılabilir. Bu field'ın değeri bir HTTP-date ya da response zamanından itibaren saniye cinsinden (ondalık) bir tam sayı olabilir.

D.2.9 Title

Title entity-header field'ı, entity'nin başlığını belirtir.

D.2.10 URI

URI entity-header field'ı, Request-URI kaynağının tanımlanabileceği Uniform Resource Identifier'ların (Bölüm 3.2) bir kısmını veya tamamını içerebilir. Belirtilen URI'ler kullanılarak kaynağa erişilebileceğine dair herhangi bir garanti yoktur.